BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ KỸ THUẬT CÔNG NGHIỆP

KHOA KHOA HỌC ỨNG DỤNG

BÁO CÁO

ỨNG DỤNG MÔ HÌNH NGÔN NGỮ LỚN (LLM) TRONG XÂY DỰNG HỆ THỐNG CHATBOT THỜI TIẾT

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU

Danh sách thành viên: Lê Quang Linh – 22174600061
Từ Tất Duy – 22174600050
Nguyễn Văn Hà – 22174600018
Lớp: DHKL16A2HN
Giảng viên hướng dẫn: ThS. Trần Quốc Hưng

HÀ NỘI – 2026

TÓM TẮT THÔNG TIN BÀI BÁO CÁO

Tên đề tài: Xây dựng hệ thống Chatbot thời tiết .

Lĩnh vực: Trí tuệ nhân tạo, xử lý ngôn ngữ tự nhiên, phát triển hệ thống chatbot.

Ngôn ngữ lập trình: Python.

Công nghệ chính: FastAPI, LangGraph, LangChain, Ollama, SQLite, WeatherAPI.

Mục tiêu hệ thống: Xây dựng một chatbot có thể hiểu câu hỏi thời tiết bằng tiếng Việt, phân loại ý định, trích xuất địa điểm, gọi dữ liệu thời tiết thực tế và sinh phản hồi tự nhiên cho người dùng.

Sản phẩm đầu ra: Hệ thống chatbot hoạt động thông qua API và có khả năng lưu lịch sử hội thoại, quản lý người dùng, hỗ trợ truy vấn thời tiết hiện tại, dự báo thời tiết, gợi ý địa điểm và small talk.

Nhận xét chung: Đề tài có tính ứng dụng thực tiễn, kết hợp giữa kiến thức xử lý ngôn ngữ tự nhiên, thiết kế backend, quản lý trạng thái hội thoại và tích hợp dịch vụ dữ liệu ngoài. Đây là một bài toán phù hợp với định hướng chuyên ngành Khoa học dữ liệu và xu hướng phát triển các hệ thống trợ lý thông minh hiện nay.

Mục lục

Danh mục hình **3**

LỜI MỞ ĐẦU **4**

1 TỔNG QUAN LÝ THUYẾT **5**

1.1 Khái niệm và vai trò của chatbot 5
1.2 Phân loại chatbot 5
1.2.1 Chatbot dựa trên luật 6
1.2.2 Chatbot dựa trên học máy 6
1.2.3 Chatbot kết hợp 7
1.3 Xử lý ngôn ngữ tự nhiên trong chatbot 7
1.3.1 Tiền xử lý câu hỏi 7
1.3.2 Phân tích ý định 7
1.3.3 Nhận diện thực thể 8
1.3.4 Sinh phản hồi 8
1.4 Mô hình ngôn ngữ lớn 8
1.5 Kiến trúc quản lý trạng thái hội thoại 8
1.6 LangGraph và mô hình đồ thị trạng thái 9
1.7 Ứng dụng Framework FastAPI trong xây dựng hệ thống Backend 10
1.8 Tích hợp dữ liệu thời tiết thông qua API 11
1.9 Vai trò của hệ quản trị cơ sở dữ liệu SQLite 11
1.10 Bài toán gợi ý địa điểm dựa trên điều kiện thời tiết 11
1.11 Kết luận chương 12

2 QUY TRÌNH XÂY DỰNG HỆ THỐNG VÀ CÁC BƯỚC THỰC HIỆN **13**

2.1 Phân tích bài toán và quy trình xây dựng nguồn dữ liệu 13
2.1.1 Yêu cầu chức năng 14
2.1.2 Yêu cầu phi chức năng 15
2.2 Thiết kế kiến trúc tổng thể 15
2.2.1 Mô tả các module chính 17
2.3 Quy trình xử lý một câu hỏi của người dùng 17
2.4 Xây dựng workflow hội thoại bằng LangGraph 18
2.5 Sơ đồ kiến trúc hệ thống 19
2.6 Thiết kế dữ liệu hội thoại 20
2.6.1 Danh sách tin nhắn 21
2.6.2 Ý định hiện tại 22
2.6.3 Dữ liệu thời tiết và dự báo 22
2.6.4 Câu hỏi chờ bổ sung 23
2.7 Xây dựng module tích hợp LLM 24

2.7.1 Phân loại ý định . . . 25
2.7.2 Trích xuất địa điểm và hoạt động . . . 26
2.7.3 Sinh đáp án tự nhiên . . . 26
2.8 Xây dựng module thời tiết . . . 27
2.9 Xây dựng module gợi ý địa điểm . . . 27
2.10 Quản lý người dùng và lịch sử hội thoại . . . 28
2.10.1 Đăng ký . . . 29
2.10.2 Đăng nhập . . . 29
2.10.3 Lưu lịch sử . . . 30
2.11 Cấu hình và môi trường chạy . . . 30
2.11.1 Khóa API của WeatherAPI . . . 31
2.11.2 Cấu hình môi trường . . . 31
2.11.3 Mô hình LLM hoặc máy chủ Ollama . . . 31
2.11.4 Tổng quan quy trình khởi chạy hệ thống . . . 32
2.11.5 Ý nghĩa của việc tổ chức cấu hình rõ ràng . . . 32
2.12 Khó khăn trong quá trình xây dựng . . . 33
2.13 Đánh giá quy trình xây dựng . . . 33
2.14 Kết luận chương . . . 33

3 KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG 35
3.1 Mục tiêu đánh giá cho hệ thống chatbot . . . 35
3.2 Môi trường thử nghiệm . . . 35
3.3 Kết quả hoạt động theo nhóm chức năng . . . 35
3.3.1 Chức năng đăng ký và đăng nhập . . . 35
3.3.2 Chức năng nhận câu hỏi và quản lý session . . . 36
3.3.3 Chức năng phân loại ý định . . . 36
3.3.4 Chức năng trích xuất địa điểm . . . 37
3.3.5 Chức năng truy xuất dữ liệu thời tiết hiện tại . . . 38
3.3.6 Chức năng truy xuất dự báo thời tiết . . . 38
3.3.7 Chức năng gợi ý địa điểm . . . 39
3.4 Bảng tổng hợp kết quả hoạt động . . . 40
3.5 Ưu điểm của hệ thống . . . 40
3.6 Hạn chế của hệ thống . . . 40
3.7 Khả năng mở rộng của hệ thống . . . 41
3.8 Nhận xét chung về kết quả . . . 41
3.9 Kết luận chương . . . 41

4 KẾT LUẬN TỔNG THỂ 42
4.1 Kết luận chung . . . 42
4.2 Giá trị đạt được của đề tài . . . 42
4.3 Những điểm thành công chính . . . 43
4.4 Những hạn chế cần khắc phục . . . 43
4.5 Hướng phát triển tương lai . . . 43
4.6 Tổng kết cuối cùng . . . 43

TÀI LIỆU THAM KHẢO 45

Danh mục hình

1.1 Sơ đồ phân loại chatbot theo phương pháp xây dựng 6
1.2 Quy trình xử lí ngôn ngữ tự nhiên 7
1.3 Sơ đồ cấu trúc quản lí hội thoại 9

2.1 Quy trình xây dựng địa danh và địa điểm gợi ý 13
2.2 Sơ đồ tổ chức module hệ thống 15
2.3 Quy trình xử lí 1 câu hỏi người dùng 17
2.4 Workflow hội thoại bằng LangGraph 18
2.5 Sơ đồ kiến trúc và luồng xử lý của hệ thống Chatbot_Weather 19
2.6 Quy trình truy xuất dữ liệu 20
2.7 Sơ đồ lưu lại lịch sử hội thoại 24

3.1 Hệ thống SQLite 35
3.2 Ví dụ kết quả 1 36
3.3 Ví dụ kết quả 2 37
3.4 Ví dụ kết quả 3 37
3.5 Ví dụ kết quả 3 38
3.6 Ví dụ kết quả 4 39

LỜI MỞ ĐẦU

Trong những năm gần đây, sự phát triển mạnh mẽ của trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên đã mở ra nhiều hướng ứng dụng thực tiễn trong đời sống và trong các hệ thống phần mềm hiện đại. Một trong những ứng dụng tiêu biểu là chatbot, tức hệ thống có khả năng tiếp nhận câu hỏi của người dùng, phân tích nội dung và tạo phản hồi dưới dạng ngôn ngữ tự nhiên. Chatbot ngày càng được sử dụng rộng rãi trong nhiều lĩnh vực như chăm sóc khách hàng, giáo dục, thương mại điện tử, tra cứu thông tin và hỗ trợ ra quyết định. So với các giao diện truyền thống, chatbot mang lại cách tương tác linh hoạt hơn, gần với thói quen giao tiếp tự nhiên của con người hơn và có khả năng phục vụ người dùng liên tục trong nhiều tình huống khác nhau.

Xuất phát từ thực tế đó, nhóm lựa chọn xây dựng đề tài chatbot thời tiết. Đây là một bài toán có tính ứng dụng cao vì nhu cầu tra cứu thời tiết, dự báo thời tiết và tham khảo các hoạt động phù hợp với điều kiện thời tiết là nhu cầu thường xuyên trong đời sống hằng ngày. Tuy nhiên, để xây dựng được một chatbot có khả năng xử lý tốt bài toán này, hệ thống không chỉ cần trả về dữ liệu thời tiết đơn thuần mà còn phải hiểu được câu hỏi của người dùng, xác định đúng ý định, trích xuất được địa điểm liên quan, duy trì ngữ cảnh hội thoại và phản hồi lại theo cách rõ ràng, tự nhiên và dễ hiểu.

Trong quá trình thực hiện đề tài, nhóm tập trung xây dựng hệ thống theo hướng có cấu trúc rõ ràng và dễ mở rộng. Hệ thống được tổ chức dựa trên nhiều thành phần phối hợp với nhau như lớp giao tiếp API, bộ quản lý trạng thái hội thoại, mô-đun xử lý ngôn ngữ, mô-đun truy xuất dữ liệu thời tiết, mô-đun gợi ý địa điểm và phần quản lý người dùng cùng lịch sử hội thoại. Đồng thời, nhóm cũng tiến hành xây dựng và chuẩn hóa dữ liệu địa danh đầu vào nhằm phục vụ cho việc nhận diện địa điểm và hỗ trợ chức năng gợi ý. Việc kết hợp giữa luồng xử lý hội thoại có trạng thái và mô hình ngôn ngữ lớn giúp chatbot không chỉ vận hành theo đúng logic nghiệp vụ mà còn có khả năng phản hồi tự nhiên hơn trong các tình huống thực tế.

Mục tiêu của đề tài là xây dựng một chatbot có thể tiếp nhận câu hỏi bằng tiếng Việt, nhận diện các nhóm yêu cầu chính liên quan đến thời tiết, truy xuất dữ liệu từ dịch vụ thời tiết bên ngoài, hỗ trợ hội thoại nhiều lượt khi thông tin chưa đầy đủ và lưu lại lịch sử tương tác của người dùng. Bên cạnh đó, hệ thống còn được mở rộng thêm chức năng gợi ý địa điểm hoặc hoạt động phù hợp với điều kiện thời tiết nhằm tăng tính hữu ích và tính ứng dụng của sản phẩm.

Báo cáo này được thực hiện nhằm trình bày toàn bộ quá trình xây dựng hệ thống, từ bước phân tích bài toán, chuẩn bị dữ liệu, thiết kế kiến trúc, xây dựng các mô-đun chức năng đến quá trình kiểm thử và đánh giá kết quả hoạt động. Nội dung báo cáo được chia thành bốn chương. Chương 1 trình bày các cơ sở lý thuyết liên quan đến chatbot, xử lý ngôn ngữ tự nhiên, mô hình ngôn ngữ lớn, quản lý trạng thái hội thoại và các công nghệ được sử dụng trong hệ thống. Chương 2 tập trung mô tả quy trình xây dựng hệ thống, cách tổ chức dữ liệu, kiến trúc các mô-đun và luồng xử lý hội thoại. Chương 3 trình bày kết quả triển khai và đánh giá hệ thống thông qua các chức năng chính, các kịch bản hội thoại tiêu biểu, cũng như những ưu điểm và hạn chế hiện tại. Cuối cùng, Chương 4 đưa ra kết luận tổng thể của đề tài và đề xuất một số hướng phát triển trong tương lai.

Chương 1

TỔNG QUAN LÝ THUYẾT

1.1 Khái niệm và vai trò của chatbot

Chatbot là một hệ thống phần mềm được xây dựng nhằm giao tiếp với con người thông qua ngôn ngữ tự nhiên. Tùy theo cách thiết kế và mục tiêu ứng dụng, chatbot có thể hoạt động trên nền tảng văn bản, giọng nói hoặc kết hợp cả hai hình thức. Về mặt bản chất, chatbot đóng vai trò như một tác nhân trung gian giữa người dùng và hệ thống thông tin, giúp con người tiếp cận dữ liệu, dịch vụ hoặc tác vụ một cách nhanh chóng và thân thiện hơn.

Vai trò của chatbot ngày càng trở nên quan trọng bởi những nguyên nhân sau:

— Chatbot có thể phục vụ liên tục 24/7 mà không phụ thuộc vào thời gian làm việc.

— Chatbot giúp giảm tải cho nhân sự ở các bộ phận hỗ trợ hoặc tư vấn.

— Chatbot có thể xử lý khối lượng lớn yêu cầu đồng thời.

— Chatbot tạo ra trải nghiệm giao tiếp tự nhiên hơn so với giao diện truyền thống.

— Chatbot có thể trở thành lớp giao tiếp thông minh cho các hệ thống dữ liệu phức tạp.

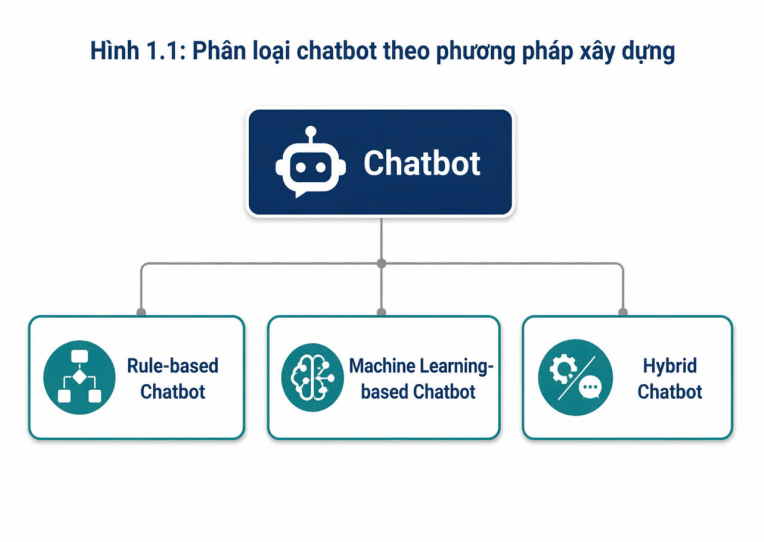
Trong bài toán thời tiết, chatbot có thể trở thành một trợ lý số giúp người dùng nhanh chóng biết được thông tin khí tượng, dự báo thời tiết, gợi ý hoạt động phù hợp hay lên kế hoạch ngắn hạn cho việc di chuyển hoặc vui chơi.

1.2 Phân loại chatbot

Xét theo cách xây dựng, chatbot thường được chia thành ba nhóm chính.
Chatbot theo luật (Rule-based): Hoạt động dựa trên các quy tắc và kịch bản có sẵn, chỉ phản hồi khi người dùng nhập đúng từ khóa.

Chatbot AI nhận diện ý định (Intent-based): Ứng dụng NLP để phân tích câu từ, hiểu ý định người dùng và truy xuất câu trả lời từ dữ liệu đã huấn luyện.

Chatbot dựa trên Mô hình ngôn ngữ lớn (LLM-based): Hiểu ngữ cảnh sâu sắc, quản lý bộ nhớ hội thoại và tự động tạo ra câu trả lời tự nhiên, linh hoạt.



Hình 1.1: Sơ đồ phân loại chatbot theo phương pháp xây dựng

1.2.1 Chatbot dựa trên luật

Đây là loại chatbot hoạt động dựa trên các quy tắc được xác định trước. Hệ thống sẽ kiểm tra đầu vào của người dùng, so khớp với một tập mẫu câu hoặc từ khóa, sau đó trả về phản hồi tương ứng.

Ưu điểm của cách tiếp cận này là:

— Dễ xây dựng.

— Dễ kiểm soát nội dung phản hồi.

— Phù hợp với bài toán đơn giản, ít biến thể.

Tuy nhiên, hạn chế là:

— Khó xử lý các cách diễn đạt đa dạng.

— Không hiểu được ngữ cảnh sâu.

— Kém linh hoạt khi hội thoại nhiều lượt.

1.2.2 Chatbot dựa trên học máy

Loại chatbot này sử dụng các mô hình học máy hoặc học sâu để nhận diện ý định, trích xuất thực thể và sinh phản hồi. Chatbot không còn phụ thuộc hoàn toàn vào luật cứng mà có thể học từ dữ liệu.

Ưu điểm:

— Linh hoạt hơn khi xử lý câu hỏi tự nhiên.

— Có khả năng tổng quát hóa từ dữ liệu huấn luyện.

— Thích hợp với các bài toán có nhiều biến thể ngôn ngữ.

1.2.3 Chatbot kết hợp

Đây là hướng tiếp cận mà đề tài sử dụng. Hệ thống kết hợp giữa luồng điều khiển có kiểm soát và năng lực hiểu ngôn ngữ của mô hình học máy hoặc mô hình ngôn ngữ lớn. Cụ thể, các bước logic như kiểm tra thiếu thông tin, gọi API, lưu trạng thái vẫn được điều phối bằng workflow rõ ràng; trong khi đó, các bước cần sự linh hoạt về ngôn ngữ như phân loại ý định, trích xuất địa điểm khó và sinh phản hồi sẽ được giao cho LLM xử lý.

Cách kết hợp này giúp tận dụng ưu điểm của cả hai phía:

— Luồng xử lý logic rõ ràng, tránh phản hồi ngẫu nhiên.

— Câu trả lời vẫn tự nhiên và linh hoạt.

— Dễ mở rộng về nghiệp vụ mà vẫn giữ được trải nghiệm người dùng tốt.

1.3 Xử lý ngôn ngữ tự nhiên trong chatbot

Xử lý ngôn ngữ tự nhiên là nền tảng cốt lõi để một chatbot có thể hoạt động hiệu quả. Trong đề tài, NLP được thể hiện ở nhiều tầng khác nhau.

Hình 1.2: Quy trình xử lí ngôn ngữ tự nhiên

1.3.1 Tiền xử lý câu hỏi

Khi người dùng gửi câu hỏi, hệ thống cần tiếp nhận và chuẩn hóa ở mức nhất định, chẳng hạn loại bỏ khoảng trắng thừa, giữ nguyên nội dung phục vụ phân tích ngữ nghĩa, và chuẩn bị văn bản cho bước suy luận sau đó.

1.3.2 Phân tích ý định

Ý định là mục đích chính của câu hỏi. Ví dụ:

— "Hà Nội hôm nay có mưa không?" → Ý định hỏi thời tiết hiện tại.

— "Ba ngày tới ở Huế như thế nào?" → Ý định dự báo.

— "Trời đẹp thì nên đi đâu chơi?" → Ý định gợi ý.

— "Xin chào" → Small talk.

Phân tích đúng ý định là điều kiện tiên quyết để chatbot chọn đúng nhánh xử lý.

1.3.3 Nhận diện thực thể

Thực thể ở đây bao gồm địa điểm, số ngày dự báo, loại hoạt động hoặc các chi tiết bổ sung khác. Ví dụ trong câu "Dự báo thời tiết Đà Lạt 2 ngày tới", hệ thống cần nhận diện "Đà Lạt" là địa điểm và "2 ngày tới" là số ngày cần dự báo.

1.3.4 Sinh phản hồi

Sau khi có dữ liệu thực tế, chatbot vẫn cần chuyển đổi dữ liệu đó sang ngôn ngữ phù hợp để người dùng dễ hiểu. Đây là bước sinh phản hồi tự nhiên, thay vì chỉ hiển thị dữ liệu thô.

1.4 Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn là các mô hình học sâu được huấn luyện trên tập dữ liệu văn bản rất lớn để học cấu trúc ngôn ngữ và quan hệ ngữ nghĩa. Những mô hình này có thể thực hiện nhiều nhiệm vụ như tóm tắt, trả lời câu hỏi, sinh văn bản, phân loại ngôn ngữ, suy luận ngữ cảnh.

Trong hệ thống chatbot thời tiết, LLM đóng vai trò ở ba nhiệm vụ chính:

— Phân loại ý định người dùng.

— Trích xuất địa điểm hoặc loại hoạt động khi phương pháp từ khóa không đủ hiệu quả.

— Sinh phản hồi cuối cùng theo phong cách tự nhiên, dễ tiếp cận.

Sở dĩ LLM phù hợp trong bài toán này vì người dùng có thể diễn đạt rất đa dạng. Chẳng hạn, cùng một mục đích nhưng có thể có các cách nói khác nhau như:

— "Hôm nay ở Hà Nội thời tiết thế nào?"

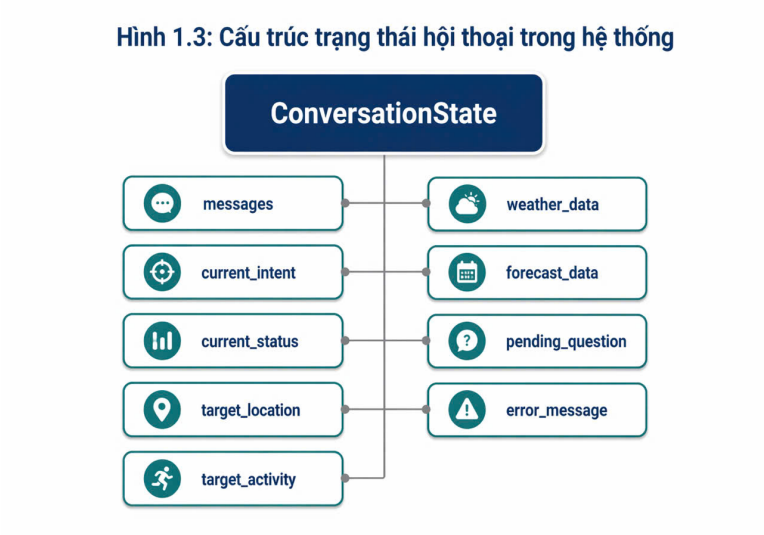
— "Ngoài Hà Nội bây giờ ra sao?"

— "Cho tôi xem tình hình thời tiết Hà Nội hôm nay."

Một hệ thống chỉ dựa trên từ khóa sẽ khó bao phủ hết các cách diễn đạt như vậy, trong khi LLM có thể linh hoạt hơn đáng kể.

1.5 Kiến trúc quản lý trạng thái hội thoại

Trong một cuộc trò chuyện thực tế, người dùng thường không cung cấp đầy đủ thông tin ngay từ đầu. Họ có thể hỏi ngắn gọn, mơ hồ hoặc bổ sung dần dữ liệu ở những lượt sau. Vì vậy, chatbot cần có khả năng quản lý trạng thái hội thoại.



Hình 1.3: Sơ đồ cấu trúc quản lí hội thoại

Trạng thái hội thoại trong đề tài được lưu bằng một cấu trúc dữ liệu có các thuộc tính như:

— `messages`: danh sách tin nhắn.

— `current_intent`: ý định hiện tại.

— `current_status`: trạng thái xử lý.

— `target_location`: địa điểm mục tiêu.

— `target_activity`: loại hoạt động.

— `weather_data`: dữ liệu thời tiết hiện tại.

— `forecast_data`: dữ liệu dự báo.

— `pending_question`: câu hỏi chờ bổ sung thông tin.

— `error_message`: thông báo lỗi nếu phát sinh.

Nhờ đó, chatbot có thể xử lý hội thoại nhiều lượt. Ví dụ:

— Người dùng: "Thời tiết hôm nay thế nào?"

— Chatbot: "Bạn muốn xem thời tiết ở đâu?"

— Người dùng: "Ở Hải Phòng."

Nếu không có cơ chế trạng thái, hệ thống sẽ không hiểu "Ở Hải Phòng" là phần bổ sung cho câu hỏi trước.

1.6 LangGraph và mô hình đồ thị trạng thái

LangGraph là công cụ hỗ trợ xây dựng ứng dụng hội thoại có trạng thái theo mô hình đồ thị. Thay vì tổ chức chương trình theo lối tuần tự đơn giản, hệ thống được chia thành các node và cạnh chuyển trạng thái. Mỗi node đại diện cho một bước xử lý riêng.

Ưu điểm của việc ứng dụng LangGraph

- **Biểu diễn logic rõ ràng:** Hỗ trợ mô hình hóa và quản lý các luồng hội thoại phức tạp, gồm nhiều bước một cách dễ dàng thông qua cấu trúc đồ thị.
- **Khả năng mở rộng linh hoạt:** Cho phép tích hợp thêm các nhánh xử lý hoặc các tính năng mới một cách độc lập mà không làm phá vỡ cấu trúc hiện tại của hệ thống.
- **Tối ưu tính trực quan:** Giúp quá trình thiết kế, theo dõi trạng thái và gỡ lỗi (debug) kiến trúc hệ thống trở nên minh bạch, dễ nắm bắt hơn.
- **Giảm thiểu độ phức tạp của mã nguồn:** Khắc phục hiệu quả tình trạng lồng ghép quá nhiều câu lệnh điều kiện (if-else), giúp mã nguồn sạch hơn và dễ bảo trì.

Luồng xử lý chính của hệ thống

Trong đề tài này, luồng hoạt động của chatbot trợ lý thời tiết và du lịch được thiết kế theo các bước sau:

1. **Tiếp nhận dữ liệu đầu vào:** Hệ thống thu nhận câu hỏi hoặc yêu cầu trực tiếp từ phía người dùng.
2. **Phân tích ý định (Intent Classification):** Đánh giá truy vấn để xác định mục đích chính của người dùng như tra cứu thời tiết, tìm kiếm địa danh hay gợi ý lịch trình.
3. **Trích xuất thực thể (Entity Extraction):** Nhận diện và bóc tách các thông tin quan trọng từ câu hỏi, đặc biệt là các thực thể liên quan đến địa điểm và thời gian.
4. **Kiểm tra tính toàn vẹn của thông tin:** Đánh giá dữ liệu hiện có để xác định xem đã đủ thông tin thực hiện tác vụ chưa, nếu thiếu sẽ kích hoạt phản hồi yêu cầu làm rõ.
5. **Truy xuất dữ liệu ngoại vi:** Thực hiện gọi các API thời tiết thời gian thực hoặc truy vấn cơ sở dữ liệu điểm đến du lịch dựa trên các thực thể đã bóc tách.
6. **Tổng hợp ngữ cảnh (Context Aggregation):** Kết hợp dữ liệu ngoại vi vừa thu thập với bối cảnh hiện tại của cuộc hội thoại để tạo ra bộ thông tin đầu vào hoàn chỉnh.
7. **Sinh phản hồi (Response Generation):** Sử dụng sức mạnh của mô hình ngôn ngữ lớn để tạo ra câu trả lời tự nhiên, chính xác và giàu thông tin cho người dùng.
8. **Cập nhật bộ nhớ (Memory Update):** Lưu trữ nội dung hội thoại vào lịch sử hệ thống để đảm bảo tính nhất quán và liền mạch cho các lượt tương tác kế tiếp.

1.7 Ứng dụng Framework FastAPI trong xây dựng hệ thống Backend

FastAPI được lựa chọn là nền tảng xây dựng backend cho hệ thống nhờ vào các đặc tính hiện đại và hiệu năng vượt trội trong hệ sinh thái Python. Những ưu điểm cốt lõi bao gồm:

- **Hiệu năng tối ưu:** Tốc độ xử lý tương đương với các framework của Go hay Node.js nhờ hỗ trợ lập trình bất đồng bộ (Asynchronous Programming).
- **Tự động hóa tài liệu:** Khả năng tự động khởi tạo giao diện tương tác API qua Swagger UI, giúp việc kiểm thử và tích hợp trở nên thuận tiện.

— **Kiểm soát dữ liệu chặt chẽ:** Sử dụng Pydantic để thực hiện kiểm tra kiểu dữ liệu (Type Hinting) và xác thực đầu vào, giảm thiểu sai sót trong quá trình vận hành.

Trong đề tài này, FastAPI đóng vai trò là "cầu nối" cung cấp các điểm cuối (Endpoints) quan trọng như: quản lý tài khoản (Đăng ký, Đăng nhập); xử lý luồng chat chính; truy xuất lịch sử hội thoại và giám sát trạng thái hoạt động của các dịch vụ. Việc triển khai FastAPI giúp chuyển đổi chatbot từ một chương trình chạy cục bộ thành một dịch vụ web hoàn chỉnh, sẵn sàng tích hợp với đa dạng giao diện người dùng.

1.8 Tích hợp dữ liệu thời tiết thông qua API

Để đảm bảo tính khách quan và độ chính xác của thông tin, hệ thống sử dụng **WeatherAPI** làm nguồn dữ liệu chính. Việc tích hợp API bên ngoài mang lại những giá trị thiết thực:

— **Dữ liệu đa dạng:** Cung cấp thông tin thời tiết hiện tại, dự báo ngắn hạn/dài hạn và khả năng chuẩn hóa tọa độ địa lý.

— **Tính cập nhật:** Đảm bảo phản hồi cho người dùng theo thời gian thực với độ trễ thấp.

— **Phạm vi rộng:** Hỗ trợ truy vấn dữ liệu thời tiết tại hầu hết các tỉnh thành và địa điểm du lịch trong nước cũng như quốc tế.

1.9 Vai trò của hệ quản trị cơ sở dữ liệu SQLite

SQLite được lựa chọn làm giải pháp lưu trữ nhờ tính gọn nhẹ và khả năng nhúng trực tiếp vào ứng dụng mà không cần cấu hình máy chủ phức tạp. Những ưu điểm chính bao gồm:

— **Tính linh hoạt:** Không đòi hỏi quy trình quản trị máy chủ chuyên biệt, dễ dàng triển khai và di động.

— **Hiệu suất:** Đáp ứng tốt tốc độ truy xuất với các tập dữ liệu có quy mô vừa và nhỏ của một hệ thống chatbot.

Trong kiến trúc hệ thống, SQLite đảm nhận nhiệm vụ lưu trữ thông tin xác thực, hồ sơ người dùng và quản lý lịch sử tương tác để duy trì bối cảnh hội thoại.

1.10 Bài toán gợi ý địa điểm dựa trên điều kiện thời tiết

Điểm khác biệt và mang tính ứng dụng cao của chatbot trong đề tài này chính là mô-đun gợi ý địa điểm thông minh. Hệ thống thực hiện quy trình hỗ trợ ra quyết định dựa trên:

— **Phân tích điều kiện thực tế:** Đối chiếu thời tiết hiện tại (nắng, mưa, nhiệt độ) với các tiêu chí hoạt động.

— **Sở thích cá nhân:** Tiếp nhận mong muốn của người dùng (vui chơi ngoài trời, nghỉ dưỡng trong nhà, v.v.).

— **Cơ sở dữ liệu tri thức:** Truy vấn các địa danh phù hợp từ tệp cấu hình hệ thống.

Sự nâng cấp này giúp chatbot chuyển đổi từ một công cụ tra cứu thông tin đơn thuần sang một hệ thống hỗ trợ ra quyết định (Decision Support System), gia tăng đáng kể trải nghiệm người dùng.

1.11 Kết luận chương

Chương này đã trình bày các cơ sở lý thuyết quan trọng phục vụ cho việc xây dựng hệ thống Chatbot_Weather. Có thể thấy rằng bài toán chatbot thời tiết không chỉ liên quan đến xử lý ngôn ngữ tự nhiên, mà còn đòi hỏi sự phối hợp giữa nhiều thành phần như quản lý trạng thái, workflow hội thoại, backend API, dịch vụ dữ liệu bên ngoài và cơ sở dữ liệu lưu trữ. Những nội dung lý thuyết này là nền tảng để triển khai phần thiết kế và xây dựng hệ thống ở chương tiếp theo.

Chương 2

QUY TRÌNH XÂY DỰNG HỆ THỐNG VÀ CÁC BƯỚC THỰC HIỆN

2.1 Phân tích bài toán và quy trình xây dựng nguồn dữ liệu

Trước khi xây dựng các module xử lý chính, nhóm tiến hành khảo sát bài toán và chuẩn bị dữ liệu đầu vào cho hệ thống. Đây là bước cần thiết nhằm xác định chatbot sẽ xử lý những loại thông tin nào, dữ liệu lấy từ đâu và cách tổ chức dữ liệu ra sao để phục vụ quá trình nhận diện địa điểm, truy xuất thời tiết và gợi ý địa danh phù hợp.

Hình 2.1: Quy trình xây dựng địa danh và địa điểm gợi ý

Về tổng thể, dữ liệu sử dụng trong hệ thống được chia thành hai nhóm chính. Nhóm thứ nhất là dữ liệu địa danh nền, dùng để nhận diện vị trí mà người dùng nhắc tới trong câu hỏi. Nhóm thứ hai là dữ liệu địa điểm gợi ý, dùng cho chức năng đề xuất nơi đi chơi hoặc hoạt động phù hợp với điều kiện thời tiết.

Đối với dữ liệu địa danh nền, nhóm xây dựng tệp `locations.json`. Tệp này lưu các thông tin cơ bản như tên địa danh, tên đầy đủ, dạng slug không dấu và tọa độ địa lý. Trong quá trình chuẩn bị

dữ liệu, nhóm thực hiện các bước gồm: thu thập danh sách địa phương, chuẩn hóa tên gọi, loại bỏ dấu tiếng Việt để tạo slug, sau đó sử dụng dịch vụ geocoding để tra cứu kinh độ và vĩ độ cho từng địa danh. Sau khi hoàn tất, dữ liệu được lưu lại thành tệp JSON có cấu trúc thống nhất để chatbot sử dụng trong quá trình vận hành.

Bên cạnh đó, nhóm xây dựng thêm tệp `Suggested_locations.json` nhằm phục vụ chức năng gợi ý. Tệp này chứa thông tin chi tiết hơn về từng địa điểm như mã địa danh, tên địa điểm, khu vực, tỉnh/thành, xã, tọa độ, nhóm hoạt động phù hợp và mô tả ngắn. Dữ liệu này giúp chatbot không chỉ trả lời về thời tiết mà còn có thể đề xuất các địa điểm phù hợp khi người dùng có nhu cầu tham khảo nơi đi chơi hoặc hoạt động tương ứng với điều kiện thời tiết.

Như vậy, quy trình chuẩn bị dữ liệu đầu vào của hệ thống có thể tóm tắt theo các bước chính như sau:

1. Thu thập danh sách địa danh và các địa điểm cần sử dụng trong hệ thống.
2. Chuẩn hóa tên gọi và tổ chức dữ liệu theo định dạng JSON.
3. Tra cứu và bổ sung tọa độ địa lý cho các địa danh.
4. Xây dựng tệp dữ liệu nền `locations.json` cho nhận diện địa điểm.
5. Xây dựng tệp `Suggested_locations.json` cho chức năng gợi ý.
6. Tích hợp các nguồn dữ liệu này vào chatbot để phục vụ xử lý hội thoại.

Việc chuẩn bị dữ liệu theo quy trình trên giúp hệ thống có nguồn dữ liệu đầu vào rõ ràng, có cấu trúc và thuận lợi cho các bước xử lý tiếp theo. Đây là nền tảng quan trọng để chatbot nhận diện đúng địa điểm, truy xuất đúng dữ liệu thời tiết và hỗ trợ gợi ý phù hợp hơn cho người dùng.

2.1.1 Yêu cầu chức năng

Hệ thống cần đáp ứng các chức năng chính sau:

- Người dùng có thể đăng ký và đăng nhập.
- Người dùng có thể gửi câu hỏi đến chatbot.
- Chatbot có thể phân tích câu hỏi và xác định ý định.
- Chatbot có thể trích xuất địa điểm từ nội dung câu hỏi.
- Chatbot có thể hỏi lại khi thông tin chưa đầy đủ.
- Chatbot có thể lấy dữ liệu thời tiết hiện tại.
- Chatbot có thể lấy dự báo thời tiết.
- Chatbot có thể gợi ý địa điểm hoặc hoạt động.
- Hệ thống có thể lưu và truy xuất lịch sử hội thoại.

2.1.2 Yêu cầu phi chức năng

Ngoài phần chức năng, hệ thống cũng cần đáp ứng một số yêu cầu phi chức năng như:

— Kiến trúc dễ mở rộng.

— Mã nguồn dễ bảo trì.

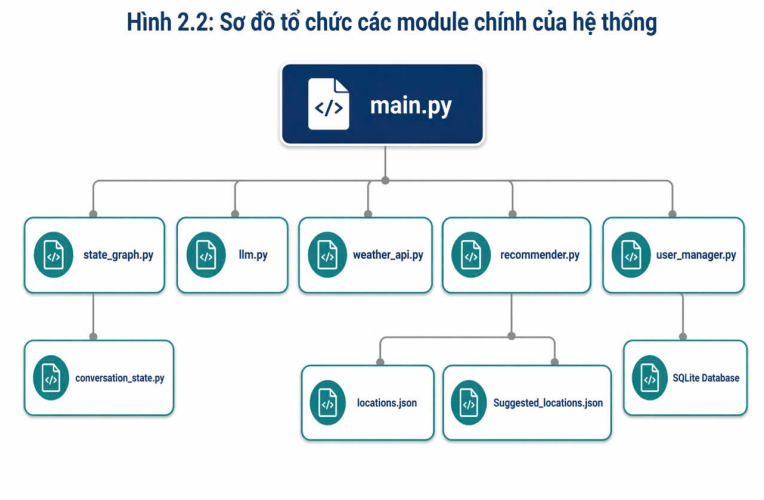
— Có khả năng kiểm thử bằng API.

— Phản hồi ở mức thời gian chấp nhận được.

— Có cơ chế xử lý lỗi cơ bản.

2.2 Thiết kế kiến trúc tổng thể

Dựa trên yêu cầu đã phân tích, hệ thống được thiết kế theo hướng module hóa. Mỗi tệp đảm nhiệm một vai trò rõ ràng, giúp việc phát triển và mở rộng thuận lợi hơn.



Hình 2.2: Sơ đồ tổ chức module hệ thống

Tệp `main.py`
Đây là điểm vào chính của hệ thống. Nhiệm vụ của file này gồm:

— Khởi tạo ứng dụng FastAPI.

— Cấu hình logging.

— Cấu hình CORS.

— Tạo các route API.

— Gọi các dịch vụ nghiệp vụ tương ứng.

— Kiểm tra tình trạng hệ thống khi khởi động.

Tệp `state_graph.py`
File này mô tả toàn bộ workflow hội thoại bằng LangGraph. Mỗi node xử lý một bước nghiệp vụ. Đây là trung tâm điều phối luồng dữ liệu.

Tệp `conversation_state.py`
File này định nghĩa các lớp liên quan đến trạng thái hội thoại như:

- Cấu trúc tin nhắn.
- Trạng thái cuộc hội thoại.
- Ý định hiện tại.
- Trạng thái xử lý.
- Session manager.

Tệp `llm.py`
Đây là nơi tích hợp mô hình ngôn ngữ lớn. File này đảm nhiệm:

- Phân loại ý định.
- Trích xuất địa điểm.
- Trích xuất loại hoạt động.
- Sinh phản hồi.
- Xử lý small talk.

Tệp `weather_api.py`
File này chịu trách nhiệm giao tiếp với WeatherAPI. Các hàm chính gồm:

- Lấy dữ liệu thời tiết hiện tại.
- Lấy dữ liệu dự báo.
- Tìm kiếm địa điểm.

Tệp `user_manager.py`
File này dùng để quản lý người dùng và lịch sử chat. Dữ liệu được lưu vào SQLite.

Tệp `recommender.py`
Đây là module gợi ý địa điểm hoặc hoạt động. Module này sử dụng dữ liệu địa điểm kết hợp với điều kiện thời tiết để lọc ra các lựa chọn phù hợp.

Tệp `locations.json`
Đây là nguồn dữ liệu địa điểm, phục vụ cho:

- Nhận diện địa danh.
- Hỗ trợ chuẩn hóa vị trí.
- Gợi ý địa điểm theo điều kiện thời tiết.

2.2.1 Mô tả các module chính

Bảng 2.1: Mô tả vai trò các module chính

STT	Module	Vai trò	Nhóm chức năng
1	`main.py`	Điểm vào của ứng dụng, khởi tạo API, định nghĩa route, điều phối các yêu cầu	API / Backend
2	`state_graph.py`	Xây dựng workflow hội thoại bằng LangGraph, kết nối các node xử lý	Điều phối hội thoại
3	`conversation_state.py`	Lưu trạng thái hội thoại, session, tin nhắn, intent, dữ liệu trung gian	Trạng thái
4	`llm.py`	Phân loại ý định, trích xuất địa điểm, sinh phản hồi tự nhiên, small talk	NLP / LLM
5	`weather_api.py`	Giao tiếp với WeatherAPI để lấy thời tiết, dự báo, tìm vị trí	Tích hợp dịch vụ ngoài
6	`recommender.py`	Lọc và gợi ý địa điểm hoặc hoạt động dựa trên điều kiện thời tiết	Gợi ý
7	`user_manager.py`	Đăng ký, đăng nhập, quản lý lịch sử hội thoại và SQLite	Người dùng / Lưu trữ
8	`locations.json`	Lưu danh sách địa danh và dữ liệu phục vụ nhận diện, gợi ý	Dữ liệu nền

2.3 Quy trình xử lý một câu hỏi của người dùng

Khi một người dùng gửi câu hỏi đến chatbot, hệ thống sẽ xử lý qua nhiều lớp, từ lớp giao tiếp API đến lớp workflow và lớp truy xuất dữ liệu.

Hình 2.3: Quy trình xử lý một câu hỏi của người dùng

Hình 2.3: Quy trình xử lí 1 câu hỏi người dùng

Được thể hiện qua các bước sau :

Bước 1: Nhận request từ client. Client gửi yêu cầu đến endpoint `/chat`. Nội dung yêu cầu thường bao gồm `user_id` để định danh người dùng và `message` để chứa nội dung câu hỏi. Đây là dữ liệu đầu vào ban đầu để hệ thống xác định ai đang tương tác và câu hỏi cần xử lý là gì.

Bước 2: Kiểm tra hoặc khởi tạo session. Sau khi nhận request, hệ thống sử dụng `SessionManager` để kiểm tra xem người dùng đã có phiên hội thoại đang hoạt động hay chưa. Nếu chưa có, một trạng thái hội thoại mới sẽ được tạo ra. Nếu đã có, hệ thống lấy lại trạng thái cũ để tiếp tục duy trì ngữ

cảnh của cuộc trò chuyện trước đó.

Bước 3: Lưu câu hỏi vào lịch sử trạng thái. Tin nhắn mới của người dùng được thêm vào danh sách `messages` trong trạng thái hội thoại. Bước này giúp chatbot lưu lại ngữ cảnh gần nhất, đồng thời tạo cơ sở cho các node xử lý phía sau có thể tham chiếu đúng thông tin vừa được gửi lên.

Bước 4: Chạy workflow LangGraph. Khi trạng thái hội thoại đã được cập nhật, hệ thống tiến hành gọi workflow xử lý bằng LangGraph. Đây là bước trung tâm của toàn bộ quá trình, nơi câu hỏi được phân tích ý định, trích xuất địa điểm, kiểm tra dữ liệu còn thiếu, sau đó điều phối sang các bước lấy dữ liệu thời tiết, dữ liệu dự báo hoặc dữ liệu gợi ý tương ứng.

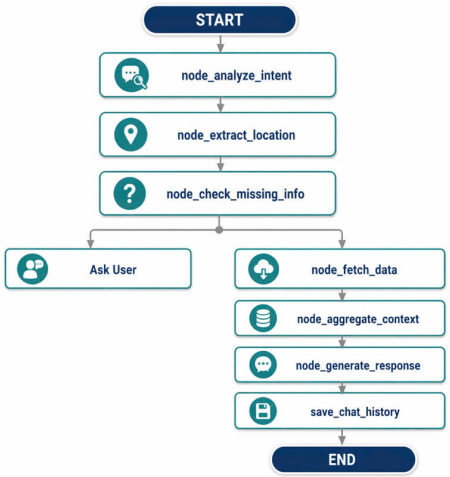
Bước 5: Sinh phản hồi và lưu cơ sở dữ liệu. Sau khi hoàn tất pipeline xử lý, chatbot sinh ra phản hồi cuối cùng dưới dạng ngôn ngữ tự nhiên. Đồng thời, nội dung hội thoại giữa người dùng và chatbot cũng được ghi vào cơ sở dữ liệu để phục vụ cho việc lưu lịch sử và tra cứu lại sau này.

Bước 6: Trả phản hồi về client. Cuối cùng, phản hồi được gửi trở lại phía client để hiển thị cho người dùng. Kết quả này là đầu ra cuối cùng của một vòng xử lý hội thoại, đồng thời vẫn gắn với trạng thái đã được cập nhật để hỗ trợ cho những lượt trao đổi tiếp theo nếu cuộc trò chuyện còn tiếp tục.

2.4 Xây dựng workflow hội thoại bằng LangGraph

Workflow hội thoại của hệ thống được xây dựng theo mô hình đồ thị trạng thái, trong đó mỗi node đảm nhiệm một chức năng riêng và phối hợp với nhau để tạo thành một luồng xử lý hoàn chỉnh. Cách tổ chức này giúp chatbot xử lý câu hỏi theo đúng trình tự, hạn chế sai sót trong quá trình điều phối và dễ mở rộng khi cần bổ sung thêm chức năng mới.

Hình 2.4: Workflow hội thoại bằng LangGraph trong Chatbot_Weather



Hình 2.4: Workflow hội thoại bằng LangGraph

Node phân tích ý định: Đây là node đầu tiên trong workflow, có nhiệm vụ tiếp nhận tin nhắn mới nhất của người dùng và xác định câu hỏi đó thuộc nhóm yêu cầu nào. Một số intent chính mà hệ thống hỗ trợ bao gồm hỏi thời tiết hiện tại, hỏi dự báo thời tiết, xin gợi ý địa điểm hoặc hoạt động, và các trường hợp hội thoại xã giao như chào hỏi hoặc cảm ơn. Kết quả của bước này có ý nghĩa rất quan trọng vì nó quyết định chatbot sẽ chuyển sang nhánh xử lý nào ở các bước tiếp theo.

Node trích xuất địa điểm: Sau khi đã xác định được intent, hệ thống tiếp tục tìm kiếm địa điểm được nhắc tới trong câu hỏi. Việc trích xuất được thực hiện theo hai lớp. Trước hết, hệ thống dò khớp với danh sách địa danh đã được lưu trong tệp `locations.json`. Nếu không tìm thấy kết quả

phù hợp, hệ thống sẽ sử dụng LLM để suy luận dựa trên ngữ cảnh câu hỏi. Cách kết hợp này giúp chatbot vừa tiết kiệm chi phí xử lý, vừa giữ được khả năng nhận diện linh hoạt với nhiều cách diễn đạt khác nhau của người dùng.

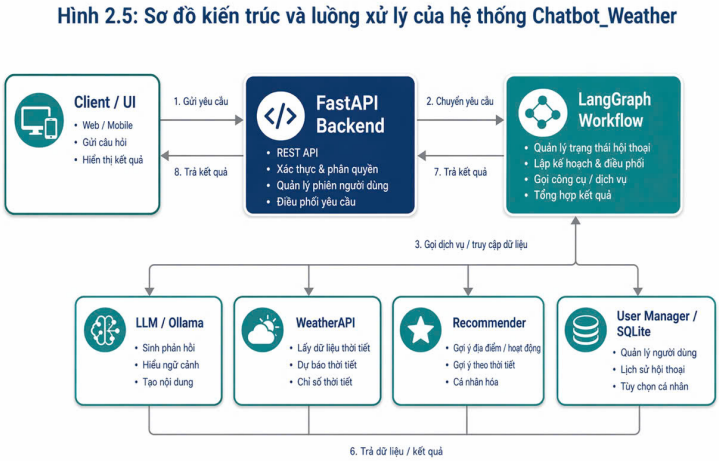
Node kiểm tra thiếu thông tin: Sau khi có intent và địa điểm, hệ thống kiểm tra xem dữ liệu đầu vào đã đủ để tiếp tục xử lý hay chưa. Nếu vẫn còn thiếu thông tin quan trọng, chatbot sẽ không trả lời ngay mà tạo câu hỏi ngược lại để yêu cầu người dùng bổ sung. Ví dụ, nếu người dùng hỏi về thời tiết nhưng chưa nêu rõ địa điểm, hệ thống sẽ hỏi lại vị trí cần xem. Nếu người dùng hỏi dự báo nhưng chưa nói rõ số ngày hoặc nơi cần dự báo, chatbot cũng sẽ yêu cầu cung cấp thêm dữ liệu. Cơ chế này giúp giảm đáng kể các trường hợp trả lời sai do suy đoán thiếu cơ sở.

Node truy xuất dữ liệu: Khi thông tin đầu vào đã đầy đủ, workflow sẽ điều hướng sang node truy xuất dữ liệu tương ứng với intent. Nếu người dùng hỏi thời tiết hiện tại, hệ thống sẽ gọi dữ liệu thời tiết hiện thời. Nếu người dùng hỏi dự báo, hệ thống sẽ lấy dữ liệu dự báo nhiều ngày. Nếu người dùng cần gợi ý địa điểm, chatbot sẽ kết hợp dữ liệu thời tiết với dữ liệu địa danh để đưa ra danh sách phù hợp. Trong kiến trúc tổng thể, đây là giai đoạn hệ thống khai thác dữ liệu từ nhiều nguồn như WeatherAPI, ForecastAPI và cơ sở dữ liệu địa điểm nội bộ.

Node tổng hợp ngữ cảnh: Sau khi dữ liệu đã được truy xuất, hệ thống tiến hành gom các thông tin cần thiết vào một ngữ cảnh thống nhất. Phần ngữ cảnh này bao gồm intent, địa điểm, dữ liệu thời tiết hoặc dự báo, cũng như các thông tin hỗ trợ khác nếu có. Đây là bước trung gian quan trọng giúp chatbot chuyển từ dữ liệu có cấu trúc sang nội dung sẵn sàng cho việc sinh phản hồi.

Node sinh phản hồi: Đây là node cuối cùng trong workflow, nơi hệ thống sử dụng LLM để diễn đạt dữ liệu thành câu trả lời hoàn chỉnh bằng ngôn ngữ tự nhiên. Tùy từng tình huống, chatbot có thể phản hồi ngắn gọn, thân thiện, nhấn mạnh những thông tin quan trọng như nhiệt độ, tình trạng thời tiết, độ ẩm hoặc khả năng mưa, đồng thời đưa ra gợi ý bổ sung nếu thấy phù hợp. Nhờ bước này, phản hồi cuối cùng không còn là dữ liệu thô mà trở thành một câu trả lời dễ hiểu, gần gũi và có tính hội thoại hơn với người dùng.

2.5 Sơ đồ kiến trúc hệ thống



Hình 2.5: Sơ đồ kiến trúc và luồng xử lý của hệ thống Chatbot_Weather

Sơ đồ cho thấy tiếp nhận & Xác thực: Người dùng gửi yêu cầu từ giao diện (Client). FastAPI đóng vai trò "cổng bảo vệ", thực hiện xác thực người dùng và chuyển yêu cầu vào hệ thống xử lý.

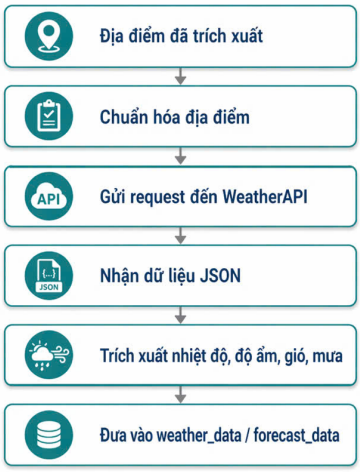
Điều phối thông minh (Trọng tâm): LangGraph đóng vai trò "nhạc trưởng", điều phối luồng công việc. Nó quyết định khi nào cần gọi WeatherAPI để lấy dữ liệu thực tế, truy xuất lịch sử từ SQLite, hay dùng LLM để hiểu ý định người dùng.

Xử lý & Phản hồi: Dữ liệu thời tiết được đưa qua bộ Recommender để đưa ra gợi ý hành động. Cuối cùng, LLM tổng hợp tất cả thông tin thành một câu trả lời tự nhiên và trả ngược lại cho người dùng thông qua FastAPI.

2.6 Thiết kế dữ liệu hội thoại

Trong một hệ thống chatbot hiện đại, đặc biệt là chatbot có khả năng hội thoại nhiều lượt như Chatbot_Weather, việc thiết kế dữ liệu hội thoại đóng vai trò trung tâm trong toàn bộ kiến trúc xử lý. Nếu coi mô hình ngôn ngữ lớn là thành phần giúp hệ thống hiểu và diễn đạt ngôn ngữ tự nhiên, thì trạng thái hội thoại chính là “bộ nhớ làm việc” giúp chatbot biết mình đang ở đâu trong tiến trình xử lý, đã nhận được thông tin gì, còn thiếu dữ liệu gì và cần thực hiện bước tiếp theo như thế nào. Nói cách khác, chất lượng của trải nghiệm hội thoại không chỉ phụ thuộc vào khả năng sinh ngôn ngữ, mà còn phụ thuộc rất lớn vào cách hệ thống lưu trữ, tổ chức và cập nhật dữ liệu hội thoại trong suốt quá trình tương tác với người dùng.

Hình 2.6: Quy trình truy xuất dữ liệu từ WeatherAPI



Hình 2.6: Quy trình truy xuất dữ liệu

Đối với bài toán chatbot thời tiết, dữ liệu hội thoại không chỉ đơn thuần là tập hợp các câu hỏi và câu trả lời. Hệ thống còn phải theo dõi nhiều thông tin trung gian khác như ý định hiện tại của người dùng, địa điểm đang được nhắc đến, dữ liệu thời tiết đã lấy được từ API, dữ liệu dự báo cho các ngày tiếp theo, trạng thái xử lý hiện tại, cũng như những câu hỏi bổ sung cần đặt ra trong trường hợp dữ liệu đầu vào chưa đầy đủ. Chính vì vậy, nhóm xây dựng một cấu trúc trạng thái hội thoại tương đối đầy đủ, vừa đảm bảo hỗ trợ được luồng xử lý hiện tại, vừa tạo điều kiện mở rộng cho các chức năng sau này.

Một trạng thái hội thoại được thiết kế tốt cần đáp ứng nhiều yêu cầu đồng thời. Thứ nhất, nó phải đủ chi tiết để lưu giữ ngữ cảnh của phiên làm việc, tránh việc chatbot xử lý từng câu độc lập một cách rời rạc. Thứ hai, cấu trúc đó phải đủ rõ ràng để các node trong workflow LangGraph có thể dễ dàng truy cập và cập nhật thông tin khi cần. Thứ ba, trạng thái hội thoại cần có tính mở rộng, tức là sau này khi thêm intent mới, thêm nguồn dữ liệu mới hoặc thêm loại phản hồi mới, hệ thống vẫn có thể bổ sung trường dữ liệu tương ứng mà không làm phá vỡ toàn bộ kiến trúc cũ.

Trong hệ thống Chatbot_Weather, trạng thái hội thoại được tổ chức xoay quanh một số nhóm thông tin chính: danh sách tin nhắn, ý định hiện tại, dữ liệu thời tiết và dự báo, trạng thái xử lý, các thực thể được trích xuất như địa điểm hoặc loại hoạt động, cũng như câu hỏi chờ bổ sung khi thiếu dữ liệu. Việc tách biệt những thành phần này giúp hệ thống không chỉ nhớ được nội dung trao đổi mà còn hiểu được ý nghĩa nghiệp vụ của từng bước hội thoại.

2.6.1 Danh sách tin nhắn

Danh sách tin nhắn là thành phần nền tảng trong trạng thái hội thoại. Đây là nơi lưu lại toàn bộ nội dung trao đổi giữa người dùng và chatbot trong một phiên làm việc. Mỗi tin nhắn không chỉ mang vai trò là dữ liệu lịch sử, mà còn là cơ sở để mô hình ngôn ngữ và các node xử lý sau đó xác định được ngữ cảnh gần nhất của cuộc trò chuyện.

Trong hệ thống, danh sách tin nhắn thường bao gồm hai vai trò chính:

- Tin nhắn từ người dùng.
- Tin nhắn phản hồi từ chatbot.

Tuy nhiên, nếu chỉ lưu nội dung văn bản đơn thuần thì chưa đủ để phục vụ cho một hệ thống hội thoại thông minh. Vì vậy, mỗi bản ghi tin nhắn còn có thể đi kèm nhiều thuộc tính bổ sung như:

- **Vai trò (role):** xác định đây là tin nhắn của người dùng hay của chatbot.
- **Nội dung (content):** phần văn bản thực tế mà hai bên trao đổi.
- **Thời gian (timestamp):** thời điểm phát sinh tin nhắn, phục vụ cho việc theo dõi lịch sử hoặc sắp xếp hội thoại.
- **Ý định liên quan (intent):** trong một số trường hợp, hệ thống có thể lưu luôn ý định được suy ra từ tin nhắn đó.
- **Trạng thái xử lý đi kèm:** cho biết tin nhắn đó thuộc giai đoạn nào trong luồng nghiệp vụ.

Việc lưu danh sách tin nhắn có ý nghĩa ở nhiều mức độ. Trước hết, nó giúp chatbot có thể "nhớ" được diễn biến hội thoại gần nhất. Ví dụ, nếu người dùng hỏi: "Thời tiết hôm nay thế nào?", chatbot sẽ phản hồi: "Bạn muốn xem thời tiết ở đâu?". Khi người dùng tiếp tục trả lời "Ở Hà Nội", hệ thống cần hiểu rằng "Ở Hà Nội" không phải là một câu hỏi độc lập, mà là phần thông tin còn thiếu của câu hỏi trước đó. Nếu không có danh sách tin nhắn để đối chiếu ngữ cảnh, chatbot rất dễ hiểu sai ý nghĩa của lượt trả lời này.

Bên cạnh đó, danh sách tin nhắn còn giúp hệ thống cải thiện chất lượng sinh phản hồi. Khi LLM được cung cấp một phần lịch sử hội thoại phù hợp, nó có thể trả lời tự nhiên hơn, tránh lặp lại thông tin, đồng thời giữ được tính liên kết giữa các lượt trao đổi. Điều này đặc biệt quan trọng trong các cuộc hội thoại nhiều bước, nơi người dùng có thể bổ sung thông tin theo từng lượt chứ không cung cấp đầy đủ ngay từ đầu.

Ngoài giá trị trong xử lý tức thời, danh sách tin nhắn còn đóng vai trò quan trọng trong việc lưu lịch sử tương tác vào cơ sở dữ liệu. Khi hệ thống lưu lại toàn bộ hội thoại, nhóm có thể sử dụng dữ liệu đó để phân tích hành vi người dùng, xác định những lỗi thường gặp, đánh giá chất lượng trả lời của chatbot và làm cơ sở cho các đợt nâng cấp sau này. Do đó, danh sách tin nhắn không chỉ là dữ liệu vận hành tạm thời, mà còn là tài sản dữ liệu có giá trị cho việc phát triển hệ thống về lâu dài.

2.6.2 Ý định hiện tại

Ý định hiện tại là một trường dữ liệu đặc biệt quan trọng trong trạng thái hội thoại, bởi nó phản ánh trực tiếp mục tiêu mà người dùng đang hướng tới tại thời điểm tương tác. Trong bài toán chatbot thời tiết, người dùng có thể đặt ra nhiều loại yêu cầu khác nhau như hỏi thời tiết hiện tại, hỏi dự báo thời tiết, xin gợi ý địa điểm, hoặc đơn giản là thực hiện small talk. Nếu hệ thống không xác định đúng ý định, toàn bộ các bước xử lý phía sau rất dễ đi sai hướng.

Việc lưu ý định hiện tại vào trạng thái hội thoại mang lại nhiều lợi ích. Trước hết, nó giúp hệ thống duy trì tính nhất quán giữa các node trong workflow. Khi node phân tích ý định hoàn tất công việc, kết quả không chỉ dùng tạm thời tại chỗ mà được cập nhật vào trạng thái chung. Từ đó, các node tiếp theo như trích xuất địa điểm, kiểm tra thiếu thông tin, gọi API hay sinh phản hồi đều có thể căn cứ vào trường này để biết nên xử lý theo hướng nào.

Ví dụ, nếu ý định hiện tại là `weather`, hệ thống sẽ hiểu rằng người dùng cần dữ liệu thời tiết hiện tại của một địa điểm cụ thể. Nếu ý định là `forecast`, hệ thống cần truy xuất dữ liệu dự báo nhiều ngày. Nếu ý định là `recommendation`, chatbot sẽ cần kết hợp thêm thông tin về hoạt động, địa điểm và điều kiện thời tiết để đưa ra gợi ý. Trong khi đó, nếu ý định là `small_talk`, chatbot có thể bỏ qua bước gọi API thời tiết và chuyển trực tiếp sang bước sinh phản hồi tự nhiên.

Một điểm quan trọng khác là ý định hiện tại giúp chatbot xử lý tốt hội thoại nhiều lượt. Trong thực tế, có những trường hợp người dùng chỉ bổ sung dữ liệu chứ không lặp lại toàn bộ câu hỏi ban đầu. Chẳng hạn:

— Người dùng: "Cho tôi xem dự báo thời tiết."

— Chatbot: "Bạn muốn xem dự báo ở đâu?"

— Người dùng: "Đà Nẵng."

Ở lượt cuối, nếu chỉ nhìn vào câu "Đà Nẵng", hệ thống sẽ không thể suy ra đây là yêu cầu gì. Nhưng nếu trạng thái hội thoại vẫn đang giữ `current_intent = forecast`, chatbot sẽ hiểu rằng địa điểm mới được bổ sung này phải được gắn với yêu cầu dự báo thời tiết trước đó.

Ngoài ra, việc lưu ý định hiện tại còn tạo điều kiện thuận lợi cho việc kiểm thử và gỡ lỗi hệ thống. Khi một phản hồi không đúng kỳ vọng, lập trình viên có thể kiểm tra xem chatbot đã nhận diện đúng intent chưa, hay bị sai từ bước đầu tiên. Đây là một chi tiết nhỏ nhưng rất quan trọng trong quá trình phát triển những hệ thống hội thoại có nhiều nhánh logic.

2.6.3 Dữ liệu thời tiết và dự báo

Trong hệ thống Chatbot_Weather, mục tiêu cuối cùng của nhiều cuộc hội thoại là cung cấp cho người dùng những thông tin cụ thể về thời tiết hoặc dự báo trong tương lai. Vì vậy, sau khi hệ thống gọi API thành công, các dữ liệu này cần được lưu vào trạng thái hội thoại để phục vụ cho bước tổng hợp ngữ cảnh và sinh phản hồi.

Dữ liệu thời tiết hiện tại thường bao gồm nhiều thuộc tính như:

— Nhiệt độ hiện tại.

— Tình trạng bầu trời, ví dụ nắng, nhiều mây, mưa nhẹ.

— Độ ẩm.

— Tốc độ gió.

— Khả năng mưa hoặc cảm giác nhiệt độ.

Trong khi đó, dữ liệu dự báo thời tiết có thể phong phú hơn, bao gồm:

— Dự báo theo từng ngày.

— Nhiệt độ cao nhất, thấp nhất.

— Khả năng mưa trong từng giai đoạn.

— Trạng thái thời tiết nổi bật.

— Xu hướng thời tiết chung trong vài ngày tới.

Việc đưa những dữ liệu này vào trạng thái hội thoại có ý nghĩa rất rõ ràng. Thứ nhất, chatbot không cần gọi lại API nhiều lần một cách không cần thiết trong cùng một vòng xử lý. Thứ hai, các node khác có thể dễ dàng truy cập dữ liệu để sử dụng vào bước tổng hợp câu trả lời hoặc gợi ý. Thứ ba, nếu cần mở rộng sang việc giải thích sâu hơn hoặc sinh phản hồi cá nhân hóa hơn, việc đã có sẵn dữ liệu trong trạng thái sẽ giúp hệ thống linh hoạt hơn rất nhiều.

Một điểm quan trọng là dữ liệu thời tiết và dự báo trong trạng thái không nên được xem như phản hồi cuối cùng, mà chỉ là dữ liệu thô hoặc bán cấu trúc. Bản thân người dùng không cần nhìn trực tiếp toàn bộ JSON trả về từ API. Thay vào đó, hệ thống cần dùng những trường dữ liệu quan trọng nhất để tổng hợp thành câu trả lời dễ hiểu. Ví dụ, thay vì hiển thị từng khóa dữ liệu, chatbot sẽ diễn đạt thành: *"Hiện tại ở Hà Nội nhiệt độ khoảng 29 độ C, trời nhiều mây và độ ẩm ở mức khá cao."* Chính vì vậy, việc lưu dữ liệu thời tiết vào trạng thái chính là cầu nối giữa phần truy xuất dữ liệu máy móc và phần sinh phản hồi mang tính hội thoại.

Ngoài ra, đây cũng là nơi rất hữu ích để phát triển thêm các chức năng nâng cao. Chẳng hạn, nếu muốn chatbot sau này đưa ra lời khuyên như "nên mang ô", "không nên đi chơi ngoài trời", hoặc "phù hợp cho du lịch ngắn ngày", thì những suy luận đó đều phải xuất phát từ dữ liệu thời tiết đã được lưu trong trạng thái trước đó.

2.6.4 Câu hỏi chờ bổ sung

Một trong những điểm khác biệt lớn giữa chatbot hội thoại nhiều lượt và hệ thống trả lời một lần là khả năng nhận biết khi nào thông tin đầu vào chưa đủ. Trong thực tế, người dùng rất hiếm khi cung cấp đầy đủ toàn bộ dữ liệu ngay từ câu hỏi đầu tiên. Họ có thể hỏi rất ngắn gọn như "Thời tiết hôm nay thế nào?", "Dự báo mấy ngày tới ra sao?", hoặc "Nên đi đâu chơi hôm nay?". Những câu hỏi này đều có mục đích nhất định nhưng chưa đủ dữ liệu để hệ thống xử lý trọn vẹn.

Đó là lý do trạng thái hội thoại cần có trường lưu **câu hỏi chờ bổ sung**. Trường này cho biết chatbot hiện đang thiếu thông tin gì và đã đặt câu hỏi ngược lại cho người dùng như thế nào. Ví dụ:

— Thiếu địa điểm → "Bạn muốn xem thời tiết ở đâu?"

— Thiếu loại hoạt động → "Bạn muốn gợi ý theo kiểu hoạt động nào?"

— Thiếu số ngày dự báo → "Bạn muốn xem dự báo trong bao nhiêu ngày tới?"

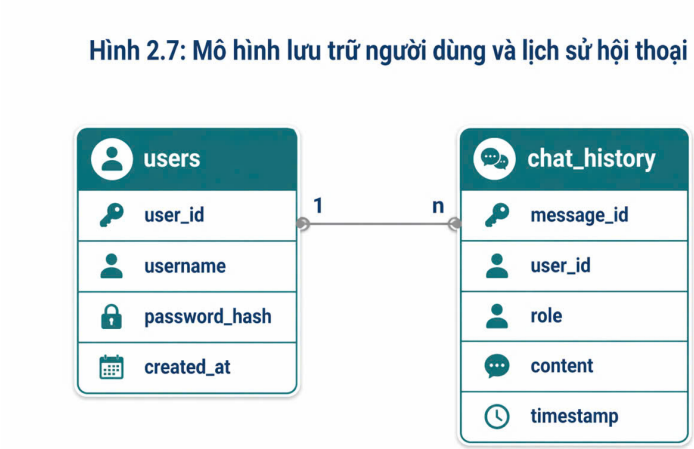
Việc lưu câu hỏi chờ bổ sung có vai trò rất quan trọng. Trước hết, nó giúp chatbot duy trì tính liên tục của hội thoại. Khi người dùng trả lời ở lượt sau, hệ thống có thể dựa vào câu hỏi đang chờ để suy ra họ đang bổ sung loại thông tin gì. Thứ hai, nó giúp tránh việc chatbot đặt lặp lại cùng một câu hỏi nếu vòng xử lý sau vẫn chưa được hoàn tất. Thứ ba, trong trường hợp kiểm thử hoặc gỡ lỗi, trường này giúp lập trình viên biết được tại sao chatbot tạm dừng ở một bước nào đó thay vì trả lời luôn.

Ở góc độ trải nghiệm người dùng, trường dữ liệu này là nền tảng để chatbot giao tiếp giống con người hơn. Một người trợ lý thực sự sẽ không cố đoán bừa khi thiếu thông tin, mà sẽ hỏi lại đúng

phần còn thiếu. Khi chatbot có khả năng làm điều đó, mức độ tự nhiên và tin cậy của hệ thống tăng lên rõ rệt. Đây cũng là một trong những yếu tố giúp Chatbot_Weather không chỉ là một công cụ tra cứu đơn giản, mà trở thành một hệ thống hội thoại có tổ chức và có ngữ cảnh.

2.7 Xây dựng module tích hợp LLM

Module `llm.py` là một trong những thành phần quan trọng nhất của toàn bộ hệ thống Chatbot_Weather. Nếu các module như `main.py`, `state_graph.py` hay `weather_api.py` đảm nhiệm vai trò điều phối luồng xử lý và truy xuất dữ liệu thực tế, thì `llm.py` chính là nơi đem lại cho chatbot khả năng "hiểu ngôn ngữ" và "diễn đạt phản hồi" theo cách gần với con người hơn. Có thể nói, đây là lớp thông minh ngôn ngữ của hệ thống.



Hình 2.7: Sơ đồ lưu lại lịch sử hội thoại

Tuy nhiên, nhóm không sử dụng LLM theo cách để nó kiểm soát hoàn toàn toàn bộ chatbot. Đây là một quyết định có chủ đích trong thiết kế kiến trúc. Bài toán thời tiết là bài toán cần độ chính xác cao về dữ liệu, bởi người dùng mong muốn nhận được thông tin đáng tin cậy về nhiệt độ, dự báo, khả năng mưa hoặc gợi ý đi lại. Nếu giao toàn bộ quyền xử lý cho mô hình ngôn ngữ, hệ thống có thể gặp rủi ro sinh ra thông tin không đúng với dữ liệu thực tế. Vì vậy, nhóm lựa chọn cách tiếp cận cân bằng hơn: sử dụng LLM ở những bước cần khả năng hiểu ngôn ngữ tự nhiên và diễn đạt linh hoạt, trong khi các bước nghiệp vụ cốt lõi như kiểm tra thiếu thông tin, gọi API, tổng hợp dữ liệu vẫn được điều phối bằng workflow rõ ràng.

Cách tiếp cận này có nhiều ưu điểm. Thứ nhất, nó tận dụng được sức mạnh của LLM trong việc xử lý các câu hỏi đa dạng, ngắn gọn, mơ hồ hoặc không theo mẫu cố định. Thứ hai, nó giữ được tính kiểm soát của hệ thống ở những phần quan trọng nhất, đặc biệt là dữ liệu thời tiết và trạng thái hội thoại. Thứ ba, nó giúp chatbot tránh được tình trạng "ảo giác" thông tin, tức là mô hình tự sinh ra dữ liệu không có thật. Nhờ đó, hệ thống vừa có tính tự nhiên trong giao tiếp, vừa có tính đáng tin cậy về nội dung.

Trong kiến trúc của hệ thống, module `llm.py` được xem như một lớp dịch vụ ngôn ngữ. Module này đảm nhiệm một số chức năng chính như:

- Phân loại ý định của câu hỏi.
- Trích xuất địa điểm hoặc loại hoạt động từ câu hỏi.

— Hỗ trợ xử lý small talk.

— Sinh câu trả lời tự nhiên dựa trên dữ liệu thực tế đã có.

Như vậy, LLM không trực tiếp quyết định toàn bộ hệ thống sẽ làm gì, nhưng nó là công cụ cực kỳ quan trọng để chuyển đổi giữa *ngôn ngữ con người* và *ngữ nghĩa nghiệp vụ* mà hệ thống có thể xử lý.

2.7.1 Phân loại ý định

Phân loại ý định là một trong những chức năng đầu tiên và quan trọng nhất của module `llm.py`. Mục đích của bước này là xác định người dùng thực sự đang muốn gì khi gửi câu hỏi. Trong bài toán chatbot thời tiết, một câu hỏi có thể thuộc nhiều nhóm khác nhau như:

— Hỏi thời tiết hiện tại tại một địa điểm.

— Hỏi dự báo thời tiết trong tương lai.

— Xin gợi ý địa điểm hoặc hoạt động phù hợp với thời tiết.

— Thực hiện small talk như chào hỏi, cảm ơn hoặc hỏi về chatbot.

Trong thực tế, người dùng không phải lúc nào cũng diễn đạt theo mẫu rõ ràng. Họ có thể dùng nhiều cách nói khác nhau cho cùng một ý định. Ví dụ:

— "Hôm nay Hà Nội có nóng không?"

— "Tình hình thời tiết ở Hà Nội bây giờ ra sao?"

— "Ngoài Hà Nội giờ thế nào rồi?"

Ba câu trên có cách diễn đạt khác nhau, nhưng đều hướng tới cùng một intent là hỏi thời tiết hiện tại. Nếu chỉ dựa trên luật cứng hoặc một vài từ khóa đơn giản, hệ thống rất dễ bỏ sót hoặc phân loại sai. Đó là lý do LLM được sử dụng ở bước này như một công cụ suy luận ngữ nghĩa.

Trong quá trình triển khai, LLM không được dùng một cách tự do hoàn toàn, mà được cung cấp ngữ cảnh và hướng dẫn rõ ràng. Điều này thường bao gồm:

— Mô tả ngắn về các intent mà hệ thống hỗ trợ.

— Yêu cầu mô hình chỉ trả về một nhóm intent phù hợp nhất.

— Các ràng buộc để tránh trả lời lan man hoặc ngoài khuôn khổ.

Việc thiết kế prompt theo hướng chặt chẽ như vậy giúp kết quả phân loại ổn định hơn và dễ tích hợp vào workflow. Sau khi LLM suy luận xong, kết quả sẽ được gán vào trường `current_intent` trong trạng thái hội thoại. Đây sẽ là dữ liệu đầu vào cho các node tiếp theo trong LangGraph.

Phân loại ý định đúng có vai trò quyết định đến toàn bộ chất lượng của hệ thống. Nếu bước này sai, chatbot có thể đi nhầm toàn bộ luồng xử lý phía sau. Ví dụ, nếu người dùng hỏi dự báo thời tiết mà chatbot hiểu nhầm thành hỏi thời tiết hiện tại, nó sẽ chỉ gọi dữ liệu hiện tại thay vì gọi dữ liệu dự báo. Do đó, dù chỉ là một trường dữ liệu trong trạng thái, `current_intent` lại là một trong những biến quan trọng nhất của cả hệ thống.

2.7.2 Trích xuất địa điểm và hoạt động

Sau khi biết được ý định của người dùng, hệ thống cần xác định các thực thể quan trọng liên quan đến câu hỏi. Trong bài toán thời tiết, thực thể quan trọng nhất thường là **địa điểm**. Ngoài ra, trong các câu hỏi dạng gợi ý, hệ thống còn cần nhận diện **loại hoạt động** mà người dùng quan tâm, chẳng hạn đi chơi, du lịch, tham quan, nghỉ ngơi hoặc hoạt động trong nhà.

Về mặt kỹ thuật, hệ thống ưu tiên một lớp xử lý đơn giản trước, chẳng hạn dò khớp địa danh với dữ liệu có sẵn trong `locations.json`. Tuy nhiên, cách này không phải lúc nào cũng đủ. Người dùng có thể viết tắt, diễn đạt địa danh theo cách không chuẩn hoặc đặt câu hỏi quá tự nhiên khiến việc đối sánh từ khóa gặp khó khăn. Ví dụ:

— "Ngoài thủ đô hôm nay sao rồi?"

— "Ở thành phố biển mấy hôm tới có mưa không?"

— "Nếu trời đẹp thì nên đi đâu dạo mát?"

Trong những tình huống như vậy, LLM đóng vai trò như một lớp suy luận phụ trợ. Mô hình có thể dựa vào ngữ cảnh của câu hỏi để nhận ra địa điểm được nhắc tới hoặc suy ra loại hoạt động mà người dùng mong muốn. Đây là một ưu thế rất lớn so với các phương pháp nhận diện thực thể thuần túy dựa trên từ điển hoặc biểu thức cố định.

Việc trích xuất địa điểm có ý nghĩa nghiệp vụ rất rõ ràng: nếu không xác định được địa điểm, hệ thống không thể gọi đúng dữ liệu thời tiết. Còn việc trích xuất loại hoạt động lại rất quan trọng trong nhánh gợi ý, bởi cùng một điều kiện thời tiết nhưng loại hoạt động khác nhau có thể dẫn tới gợi ý hoàn toàn khác nhau. Ví dụ, nếu người dùng muốn đi dạo ngoài trời thì hệ thống sẽ ưu tiên các địa điểm thoáng đãng, công viên hoặc điểm du lịch mở. Nhưng nếu người dùng muốn tránh nắng hoặc tránh mưa, hệ thống có thể nghiêng về các địa điểm trong nhà.

Từ góc độ kiến trúc, chức năng trích xuất địa điểm và hoạt động trong `llm.py` không tồn tại độc lập mà gắn chặt với trạng thái hội thoại. Kết quả sau khi suy luận sẽ được cập nhật vào các trường như `target_location` hoặc `target_activity`. Nhờ đó, các node phía sau như kiểm tra thiếu thông tin, truy xuất dữ liệu hoặc gợi ý có thể sử dụng trực tiếp mà không cần phân tích lại từ đầu.

2.7.3 Sinh đáp án tự nhiên

Sinh đáp án tự nhiên là bước thể hiện rõ nhất vai trò của LLM trong việc nâng cao trải nghiệm người dùng. Sau khi hệ thống đã hoàn thành các bước logic như xác định ý định, trích xuất thực thể, gọi API thời tiết hoặc bộ gợi ý, những gì còn lại trong trạng thái hội thoại chủ yếu là dữ liệu có cấu trúc. Tuy nhiên, dữ liệu có cấu trúc không phải là cách giao tiếp tốt với người dùng cuối. Người dùng không muốn nhìn thấy một khối JSON hay một tập giá trị rời rạc, mà mong muốn nhận được một câu trả lời dễ hiểu, gọn gàng, tự nhiên và có tính hội thoại.

Đây chính là lúc LLM phát huy thế mạnh. Từ các dữ liệu đầu vào như nhiệt độ, trạng thái trời, độ ẩm, dự báo nhiều ngày hoặc danh sách gợi ý địa điểm, mô hình sẽ diễn đạt lại thành câu trả lời hoàn chỉnh. Ví dụ, thay vì xuất ra:

— temperature: 29

— condition: cloudy

— humidity: 82

chatbot có thể trả lời:

> "Hiện tại ở Hà Nội nhiệt độ khoảng 29 độ C, trời nhiều mây và độ ẩm khá cao, nên cảm giác thời tiết có phần oi nhẹ."

Điểm quan trọng trong bước này là chatbot không được phép tự bịa thêm dữ liệu. Vì vậy, module `llm.py` được thiết kế theo hướng sinh phản hồi dựa trên dữ liệu thực tế đã có trong trạng thái, chứ không để mô hình tự sáng tác nội dung một cách tùy ý. Điều này đảm bảo tính đúng đắn của phản hồi, đồng thời vẫn giữ được tính mềm mại trong ngôn ngữ.

Ngoài ra, sinh đáp án tự nhiên còn giúp chatbot xử lý tốt hơn những tình huống cần diễn đạt khéo léo, chẳng hạn:

- Tóm tắt dự báo nhiều ngày mà không quá dài dòng.
- Đưa ra gợi ý địa điểm theo cách thân thiện.
- Phản hồi small talk một cách tự nhiên.
- Hỏi lại người dùng trong trường hợp thiếu dữ liệu nhưng vẫn giữ được cảm giác lịch sự, dễ chịu.

Có thể nói, nếu phân loại ý định là bước giúp chatbot "hiểu người dùng muốn gì", còn trích xuất thực thể là bước giúp chatbot "xác định phải xử lý dữ liệu nào", thì sinh đáp án tự nhiên chính là bước giúp chatbot "nói lại kết quả theo cách mà con người dễ tiếp nhận nhất". Đây là yếu tố then chốt làm cho Chatbot_Weather không chỉ là một công cụ truy vấn dữ liệu, mà thực sự có dáng dấp của một hệ thống hội thoại thông minh.

2.8 Xây dựng module thời tiết

Module `weather_api.py` được triển khai theo hướng bất đồng bộ. Điều này giúp hệ thống hạn chế tình trạng bị chặn trong lúc chờ phản hồi từ API bên ngoài.

Một số chức năng chính:

- `get_weather`: lấy thời tiết hiện tại.
- `get_forecast`: lấy dự báo thời tiết.
- `search_location`: tìm kiếm vị trí.

Ngoài việc gọi API, module còn phải xử lý:

- Lỗi mạng.
- API key không hợp lệ.
- Địa điểm không tìm thấy.
- Dữ liệu trả về không đầy đủ.

2.9 Xây dựng module gợi ý địa điểm

Mô-đun gợi ý địa điểm là thành phần mở rộng của hệ thống, giúp chatbot không chỉ trả lời thông tin thời tiết mà còn hỗ trợ người dùng lựa chọn địa điểm hoặc hoạt động phù hợp. Module này hoạt động bằng cách kết hợp giữa dữ liệu thời tiết hiện tại hoặc dữ báo với thông tin địa điểm đã được lưu sẵn trong hệ thống.

Về nguyên tắc, khi người dùng có nhu cầu xin gợi ý, chatbot sẽ dựa trên các yếu tố như trạng thái thời tiết, nhiệt độ và loại hoạt động mong muốn để lọc ra những địa điểm phù hợp hơn. Nhờ đó, câu trả lời của chatbot không dừng ở mức cung cấp dữ liệu, mà còn mang tính hỗ trợ ra quyết định.

Một số trường hợp gợi ý điển hình gồm:

— Nếu thời tiết thuận lợi, trời đẹp hoặc ít mưa, hệ thống có thể ưu tiên các địa điểm ngoài trời như công viên, khu tham quan hoặc nơi đi dạo.

— Nếu thời tiết xấu, có mưa hoặc điều kiện không phù hợp để di chuyển ngoài trời, chatbot sẽ ưu tiên các địa điểm trong nhà hoặc các hoạt động an toàn hơn.

— Nếu nhiệt độ ở mức mát mẻ, dễ chịu, hệ thống có thể gợi ý các hoạt động thư giãn, nghỉ ngơi hoặc du lịch ngắn.

Như vậy, module này giúp mở rộng chức năng của chatbot từ truy vấn thông tin sang hỗ trợ tư vấn. Đây là một thành phần quan trọng giúp hệ thống trở nên thực tế và hữu ích hơn đối với người dùng.

2.10 Quản lý người dùng và lịch sử hội thoại

Trong quá trình xây dựng hệ thống Chatbot_Weather, nhóm nhận thấy rằng một chatbot hoàn chỉnh không nên chỉ dừng lại ở khả năng trả lời câu hỏi. Nếu hệ thống chỉ tiếp nhận một câu hỏi, sinh một phản hồi rồi kết thúc mà không có cơ chế quản lý người dùng và lưu vết hội thoại, thì sản phẩm sẽ chỉ phù hợp ở mức thử nghiệm đơn giản. Trong khi đó, đối với một hệ thống có định hướng ứng dụng thực tế, việc quản lý tài khoản người dùng và lưu lịch sử hội thoại là những thành phần rất cần thiết. Chính vì vậy, nhóm đã tích hợp thêm module quản lý người dùng và lịch sử trò chuyện nhằm làm cho hệ thống trở nên đầy đủ hơn, có cấu trúc hơn và thuận tiện hơn trong quá trình khai thác cũng như phát triển về sau.

Phần quản lý người dùng giúp hệ thống xác định được ai đang sử dụng chatbot, từ đó gắn mỗi phiên hội thoại với một tài khoản cụ thể thay vì xử lý hoàn toàn ẩn danh. Cách làm này mang lại nhiều lợi ích. Trước hết, nó giúp chatbot có thể duy trì lịch sử trao đổi theo từng người dùng riêng biệt. Thứ hai, nó tạo cơ sở cho việc mở rộng các chức năng trong tương lai như cá nhân hóa câu trả lời, thống kê tần suất sử dụng hoặc xây dựng hồ sơ người dùng. Thứ ba, khi hệ thống được kiểm thử hoặc triển khai cho nhiều người cùng sử dụng, phần quản lý người dùng giúp tránh nhầm lẫn giữa các phiên hội thoại khác nhau.

Song song với đó, phần lưu lịch sử hội thoại đóng vai trò như một lớp dữ liệu nền giúp hệ thống không chỉ phục vụ tương tác tức thời mà còn hỗ trợ việc tra cứu lại nội dung đã trao đổi. Việc lưu lại lịch sử không chỉ có ích cho người dùng, mà còn đặc biệt quan trọng với nhóm phát triển. Nhờ lịch sử hội thoại, nhóm có thể theo dõi chatbot đã trả lời ra sao, phát hiện các trường hợp xử lý chưa tốt, xác định lỗi lặp lại nhiều lần và sử dụng những thông tin đó để cải tiến chất lượng hệ thống trong các phiên bản tiếp theo.

Trong hệ thống này, phần quản lý người dùng và lịch sử hội thoại được triển khai thông qua module `user_manager.py`, kết hợp với cơ sở dữ liệu SQLite. Việc sử dụng SQLite là phù hợp với quy mô của đề tài vì đây là hệ quản trị cơ sở dữ liệu gọn nhẹ, dễ triển khai, không yêu cầu máy chủ riêng và vẫn đáp ứng tốt nhu cầu lưu trữ người dùng cùng lịch sử hội thoại trong môi trường phát triển và thử nghiệm.

2.10.1 Đăng ký

Chức năng đăng ký được xây dựng nhằm cho phép người dùng tạo tài khoản mới trước khi sử dụng đầy đủ hệ thống. Đây là bước đầu tiên trong quy trình quản lý người dùng, giúp chatbot có thể nhận diện và phân biệt các đối tượng sử dụng khác nhau.

Về mặt quy trình, khi một người dùng mới gửi yêu cầu đăng ký thông qua API, hệ thống sẽ tiếp nhận thông tin cần thiết, sau đó kiểm tra xem tài khoản đó đã tồn tại trong cơ sở dữ liệu hay chưa. Nếu thông tin hợp lệ và chưa bị trùng lặp, hệ thống sẽ tạo bản ghi mới trong cơ sở dữ liệu SQLite. Từ thời điểm đó, người dùng đã có một danh tính rõ ràng trong hệ thống và có thể tiếp tục thực hiện các bước tiếp theo như đăng nhập và bắt đầu hội thoại với chatbot.

Việc xây dựng chức năng đăng ký không chỉ phục vụ mục đích quản trị tài khoản, mà còn giúp hệ thống có nền tảng để mở rộng trong tương lai. Chẳng hạn, khi mỗi người dùng đã có định danh riêng, chatbot có thể phát triển thêm các tính năng như lưu lịch sử theo cá nhân, ghi nhớ sở thích truy vấn thường gặp hoặc thống kê mức độ sử dụng theo từng tài khoản. Đây là điều không thể thực hiện được nếu toàn bộ hệ thống chỉ hoạt động ở chế độ ẩn danh.

Ngoài ra, chức năng đăng ký còn giúp sản phẩm trở nên hoàn chỉnh hơn dưới góc độ một hệ thống thực tế. Trong nhiều báo cáo học thuật hoặc sản phẩm thử nghiệm, phần chatbot thường chỉ tập trung vào câu hỏi và câu trả lời, trong khi bỏ qua lớp người dùng. Tuy nhiên, nhóm nhận thấy rằng việc bổ sung bước đăng ký không chỉ làm tăng tính đầy đủ của đề tài mà còn thể hiện tư duy thiết kế hệ thống một cách có tổ chức và có khả năng ứng dụng cao hơn.

2.10.2 Đăng nhập

Sau khi người dùng đã có tài khoản, bước tiếp theo là đăng nhập vào hệ thống để bắt đầu sử dụng chatbot dưới danh tính của mình. Đây là bước đóng vai trò kết nối giữa lớp quản lý người dùng và lớp xử lý hội thoại.

Khi người dùng gửi thông tin đăng nhập thông qua API, hệ thống sẽ kiểm tra dữ liệu đầu vào với những bản ghi đã lưu trong cơ sở dữ liệu. Nếu thông tin xác thực là hợp lệ, hệ thống cho phép người dùng truy cập và tiếp tục sử dụng các chức năng của chatbot. Từ thời điểm này, mỗi yêu cầu chat gửi lên sẽ được gắn với `user_id` tương ứng, nhờ đó hệ thống biết chính xác ai là người đang thực hiện hội thoại.

Việc đăng nhập có ý nghĩa rất quan trọng trong việc tổ chức luồng dữ liệu. Thay vì mọi cuộc trò chuyện đều bị trộn lẫn hoặc không rõ nguồn gốc, hệ thống có thể liên kết từng phiên trao đổi với một người dùng cụ thể. Điều này đặc biệt hữu ích khi chatbot cần lưu lịch sử trò chuyện, truy xuất lại các cuộc hội thoại trước đó, hoặc duy trì session theo từng tài khoản. Nếu không có bước đăng nhập, hệ thống sẽ khó tổ chức dữ liệu hội thoại theo người dùng một cách rõ ràng và chính xác.

Một ý nghĩa khác của chức năng đăng nhập là tăng tính liên tục trong trải nghiệm sử dụng. Người dùng sau khi đăng nhập có thể tiếp tục hội thoại với chatbot mà không bị tách rời hoàn toàn khỏi các tương tác trước đó. Mặc dù ở phiên bản hiện tại hệ thống mới chủ yếu sử dụng quản lý session trong bộ nhớ và lưu lịch sử vào SQLite, nhưng đây đã là một nền tảng quan trọng để mở rộng sang các cơ chế quản lý người dùng phức tạp hơn trong tương lai.

Nhìn chung, đăng nhập không chỉ là bước xác thực đơn thuần, mà còn là cơ chế giúp liên kết người dùng với trạng thái hội thoại, với dữ liệu lịch sử và với các chức năng mở rộng của hệ thống. Đây là lý do nhóm đưa phần này vào như một thành phần không thể thiếu trong kiến trúc tổng thể của Chatbot_Weather.

2.10.3 Lưu lịch sử

Lưu lịch sử hội thoại là một trong những thành phần quan trọng nhất giúp hệ thống chatbot trở nên có chiều sâu hơn về mặt vận hành. Trong quá trình tương tác, mỗi tin nhắn của người dùng và mỗi câu trả lời từ chatbot đều được ghi lại trong cơ sở dữ liệu. Điều này có nghĩa là hệ thống không chỉ phản hồi trong thời gian thực, mà còn duy trì được một bản ghi của toàn bộ quá trình trò chuyện.

Về mặt kỹ thuật, sau khi một vòng xử lý hội thoại kết thúc, nội dung liên quan sẽ được chuyển từ trạng thái hội thoại tạm thời sang lớp lưu trữ lâu dài trong SQLite. Tùy theo cách tổ chức dữ liệu, mỗi bản ghi có thể bao gồm các thông tin như:

— Người gửi tin nhắn là ai.

— Nội dung tin nhắn là gì.

— Thời điểm phát sinh tin nhắn.

— Ý định liên quan nếu có.

— Tài khoản người dùng sở hữu cuộc hội thoại đó.

Việc lưu lịch sử hội thoại mang lại nhiều lợi ích rõ ràng. Trước hết, người dùng có thể xem lại các nội dung đã trao đổi với chatbot trước đó. Đây là một chức năng hữu ích trong thực tế, đặc biệt khi người dùng muốn tra cứu lại thông tin thời tiết đã xem, hoặc kiểm tra lại gợi ý mà chatbot từng đưa ra.

Thứ hai, lịch sử hội thoại là nguồn dữ liệu quan trọng để phân tích quá trình sử dụng hệ thống. Nhóm phát triển có thể quan sát xem người dùng thường hỏi những dạng câu nào, intent nào xuất hiện nhiều nhất, chatbot xử lý tốt ở đâu và còn hạn chế ở đâu. Điều này đặc biệt có giá trị trong giai đoạn kiểm thử và hoàn thiện sản phẩm, bởi vì nhiều lỗi hoặc điểm chưa hợp lý chỉ bộc lộ rõ khi xem lại toàn bộ tiến trình hội thoại thay vì chỉ nhìn vào một phản hồi đơn lẻ.

Thứ ba, lịch sử hội thoại là nền tảng hỗ trợ cải tiến hệ thống trong tương lai. Chẳng hạn, nếu nhóm muốn xây dựng thêm bộ dữ liệu đánh giá chất lượng chatbot, hoặc muốn huấn luyện, tinh chỉnh prompt, cải thiện cách phân loại ý định hay cách sinh phản hồi, thì những cuộc hội thoại đã lưu chính là dữ liệu tham chiếu rất có giá trị. Có thể nói, mỗi cuộc trao đổi được lưu lại không chỉ phục vụ người dùng ở thời điểm hiện tại mà còn góp phần giúp hệ thống tốt hơn ở những phiên bản sau.

Ngoài ra, việc lưu lịch sử cũng giúp hệ thống minh bạch và dễ kiểm tra hơn. Khi xảy ra lỗi hoặc khi chatbot trả lời chưa đúng, nhóm có thể truy ngược lại hội thoại để xem dữ liệu đầu vào là gì, chatbot đã hiểu intent ra sao và phản hồi cuối cùng được sinh ra như thế nào. Đây là một ưu điểm rất lớn trong quá trình debug, bảo trì và đánh giá sản phẩm.

Tóm lại, chức năng lưu lịch sử hội thoại không chỉ đơn giản là "ghi lại tin nhắn", mà là một thành phần quan trọng hỗ trợ cả trải nghiệm người dùng, quản lý hệ thống và định hướng phát triển sản phẩm lâu dài.

2.11 Cấu hình và môi trường chạy

Để hệ thống Chatbot_Weather có thể vận hành ổn định, ngoài phần mã nguồn và các module chức năng, một yếu tố rất quan trọng khác là cấu hình môi trường chạy. Nếu ví phần mã nguồn là phần "logic" của hệ thống, thì cấu hình chính là điều kiện nền để logic đó có thể được kích hoạt đúng cách. Trong quá trình triển khai, nhóm nhận thấy rằng một hệ thống chatbot tích hợp nhiều thành phần như API backend, mô hình ngôn ngữ lớn và dịch vụ dữ liệu bên ngoài sẽ chỉ hoạt động ổn định khi các tham số cấu hình được thiết lập đầy đủ và nhất quán.

Về tổng thể, môi trường chạy của hệ thống bao gồm nhiều lớp khác nhau. Lớp đầu tiên là môi trường Python cùng các thư viện phụ thuộc cần thiết như FastAPI, LangGraph, LangChain, SQLite, `httpx` và các thư viện liên quan đến mô hình ngôn ngữ. Lớp thứ hai là các biến môi trường hoặc tệp cấu hình giúp hệ thống biết phải kết nối tới dịch vụ nào, sử dụng khóa truy cập nào và khởi tạo mô hình nào. Lớp thứ ba là các tài nguyên ngoài hệ thống như WeatherAPI hoặc máy chủ Ollama phục vụ cho việc suy luận mô hình.

Trong quá trình chạy thực tế, nếu thiếu một trong các thành phần cấu hình quan trọng thì chatbot có thể không hoạt động đúng, hoặc thậm chí không thể khởi động. Do đó, nhóm xây dựng phần cấu hình theo hướng rõ ràng, tách biệt với phần logic xử lý để thuận tiện cho việc kiểm tra, thay đổi và mở rộng.

2.11.1 Khóa API của WeatherAPI

Một trong những thành phần bắt buộc nhất của hệ thống là khóa truy cập WeatherAPI. Đây là thông tin xác thực giúp ứng dụng có quyền gửi yêu cầu đến dịch vụ thời tiết bên ngoài và nhận lại dữ liệu thời tiết hiện tại, dữ liệu dự báo hoặc kết quả tìm kiếm địa điểm.

Về quy trình sử dụng, khi người dùng gửi câu hỏi liên quan đến thời tiết, chatbot sau khi phân tích ý định và xác định địa điểm sẽ gọi đến module `weather_api.py`. Tại đây, hệ thống cần có khóa API hợp lệ để gửi request đến WeatherAPI. Nếu khóa bị thiếu, bị sai hoặc hết hiệu lực, yêu cầu sẽ thất bại và chatbot sẽ không thể lấy được dữ liệu thực tế.

Việc tách khóa API ra khỏi mã nguồn là một lựa chọn đúng đắn về mặt kỹ thuật. Thay vì viết trực tiếp khóa vào code, nhóm lưu thông tin này trong tệp môi trường hoặc biến môi trường. Cách làm này vừa giúp bảo mật tốt hơn, vừa thuận lợi khi chuyển hệ thống sang một máy khác hoặc triển khai trong môi trường mới. Chỉ cần thay đổi giá trị cấu hình, hệ thống có thể tiếp tục hoạt động mà không cần sửa logic chương trình.

Ngoài ra, việc quản lý khóa API rõ ràng còn hỗ trợ tốt cho khâu kiểm thử. Khi hệ thống không nhận được dữ liệu thời tiết, nhóm có thể nhanh chóng kiểm tra xem nguyên nhân đến từ mạng, từ API hay từ chính khóa truy cập. Điều này giúp quá trình debug hiệu quả hơn rất nhiều.

2.11.2 Cấu hình môi trường

Bên cạnh khóa API, hệ thống còn cần một số cấu hình môi trường khác để xác định cách thức vận hành. Đây có thể là những tham số như chế độ phát triển hay sản xuất, địa chỉ chạy của ứng dụng, cổng dịch vụ, mức độ ghi log hoặc đường dẫn đến các tài nguyên cần thiết.

Vai trò của cấu hình môi trường là giúp hệ thống có thể thích nghi với nhiều điều kiện triển khai khác nhau mà không cần thay đổi trực tiếp trong mã nguồn. Ví dụ, trong quá trình phát triển cục bộ, nhóm có thể chạy chatbot trên máy cá nhân với chế độ debug để quan sát log chi tiết. Nhưng khi cần trình diễn hoặc chia sẻ cho người khác truy cập, hệ thống có thể cần chuyển sang cấu hình khác, ví dụ đổi cổng chạy, bật ngrok hoặc thay đổi mức log phù hợp hơn.

Việc tách lớp cấu hình ra riêng còn giúp báo cáo trở nên rõ ràng hơn về mặt kiến trúc. Người đọc có thể hiểu rằng hệ thống không được xây dựng theo kiểu "viết cứng" mọi tham số vào mã nguồn, mà đã có tư duy tổ chức thành các lớp riêng biệt: lớp xử lý nghiệp vụ, lớp quản lý dữ liệu và lớp cấu hình môi trường. Đây là một điểm cộng lớn về mặt thiết kế phần mềm.

2.11.3 Mô hình LLM hoặc máy chủ Ollama

Một thành phần không thể thiếu khác trong môi trường chạy của hệ thống là mô hình ngôn ngữ lớn hoặc máy chủ Ollama dùng để phục vụ suy luận. Trong Chatbot_Weather, LLM không đóng vai

trò cung cấp dữ liệu thời tiết, nhưng lại cực kỳ quan trọng ở các bước xử lý ngôn ngữ như phân loại ý định, trích xuất địa điểm, nhận diện hoạt động và sinh phản hồi cuối cùng.

Để sử dụng được LLM, hệ thống cần xác định rõ mô hình nào sẽ được gọi, mô hình đó chạy cục bộ hay thông qua máy chủ trung gian, và địa chỉ kết nối tới dịch vụ suy luận là gì. Nếu các tham số này không được cấu hình đúng, module `llm.py` sẽ không thể hoạt động, kéo theo nhiều chức năng quan trọng của chatbot bị ảnh hưởng.

Trong quá trình chạy, khi một câu hỏi được gửi đến, chatbot có thể chuyển nội dung sang mô hình để phân tích ngôn ngữ tự nhiên. Nếu mô hình hoặc máy chủ Ollama chưa được khởi động, phản hồi từ hệ thống có thể bị chậm hoặc không trả về kết quả. Vì vậy, việc cấu hình và kiểm tra lớp mô hình ngôn ngữ là bước rất quan trọng trước khi vận hành chatbot.

Điểm đáng chú ý là nhóm sử dụng LLM theo hướng hỗ trợ thay vì phụ thuộc hoàn toàn. Điều này đồng nghĩa với việc mô hình vẫn là thành phần quan trọng, nhưng nó được đặt trong một kiến trúc có kiểm soát. Nói cách khác, môi trường chạy của LLM phải được chuẩn bị tốt, nhưng hệ thống vẫn giữ được sự ổn định nhờ workflow logic và các lớp dữ liệu trung gian.

2.11.4 Tổng quan quy trình khởi chạy hệ thống

Để người đọc dễ hình dung hơn, có thể tóm tắt quy trình chuẩn bị và khởi chạy hệ thống như sau:

1. Cài đặt môi trường Python và các thư viện phụ thuộc.
2. Chuẩn bị tệp cấu hình hoặc biến môi trường cần thiết.
3. Khai báo khóa truy cập WeatherAPI.
4. Kiểm tra mô hình LLM hoặc máy chủ Ollama đã sẵn sàng chưa.
5. Khởi động ứng dụng FastAPI.
6. Gửi request thử nghiệm qua endpoint chat để kiểm tra toàn bộ pipeline.

Khi tất cả các thành phần trên đều được cấu hình đúng, hệ thống sẽ có thể hoạt động ổn định: nhận câu hỏi từ người dùng, phân tích ý định, gọi dịch vụ thời tiết, sinh phản hồi tự nhiên và lưu lịch sử hội thoại.

2.11.5 Ý nghĩa của việc tổ chức cấu hình rõ ràng

Việc trình bày rõ cấu hình và môi trường chạy trong báo cáo không chỉ mang tính kỹ thuật mà còn giúp người đọc hiểu được hệ thống vận hành như thế nào trong thực tế. Nhiều sản phẩm phần mềm có thể viết được logic xử lý khá tốt, nhưng khi triển khai lại gặp khó khăn chỉ vì thiếu tổ chức ở phần cấu hình. Đối với Chatbot_Weather, nhóm đã cố gắng tách phần cấu hình ra thành một lớp riêng, từ đó làm cho hệ thống:

- Dễ kiểm tra lỗi hơn.
- Dễ chuyển đổi giữa các môi trường chạy khác nhau.
- Dễ bảo trì và nâng cấp.
- Thuận tiện hơn khi chia sẻ hoặc triển khai trên máy khác.

Như vậy, cấu hình và môi trường chạy không phải là phần phụ, mà là một phần rất quan trọng để biến hệ thống từ một đoạn mã thử nghiệm thành một sản phẩm có khả năng vận hành thực tế. Ngoài ra, nhóm cũng tổ chức các script hỗ trợ chạy hệ thống bằng `uvicorn`, kiểm thử nội bộ hoặc mở cổng qua ngrok nếu cần.

2.12 Khó khăn trong quá trình xây dựng

Trong quá trình triển khai hệ thống, nhóm gặp một số khó khăn điển hình:

— Khó cân bằng giữa việc dùng luật và dùng LLM.

— Cần thiết kế trạng thái hội thoại đủ linh hoạt nhưng không quá phức tạp.

— Phải xử lý lỗi phát sinh từ API bên ngoài.

— Cần tránh để chatbot trả lời sai khi thông tin đầu vào chưa đầy đủ.

— Việc tổ chức luồng LangGraph đòi hỏi phải xác định rõ trách nhiệm của từng node.

2.13 Đánh giá quy trình xây dựng

Quy trình triển khai của nhóm đi từ phần nền tảng đến phần thông minh:

1. Xây dựng API và bộ khung backend.
2. Thiết kế trạng thái hội thoại.
3. Tổ chức workflow bằng LangGraph.
4. Tích hợp LLM.
5. Kết nối API thời tiết.
6. Thêm chức năng gợi ý.
7. Hoàn thiện lưu trữ và quản lý người dùng.

Cách làm này giúp hệ thống phát triển từng bước, dễ kiểm thử và sửa lỗi hơn so với việc ghép nối tất cả cùng một lúc.

2.14 Kết luận chương

Trong chương này, nhóm đã trình bày toàn bộ quy trình xây dựng hệ thống Chatbot_Weather theo hướng đi từ bài toán thực tế đến thiết kế và triển khai kỹ thuật. Trước hết, nhóm xác định rõ nhu cầu của hệ thống, chuẩn bị dữ liệu đầu vào và tổ chức nguồn dữ liệu địa danh nhằm phục vụ cho việc nhận diện vị trí, truy xuất thông tin thời tiết và hỗ trợ gợi ý địa điểm. Trên cơ sở đó, hệ thống được thiết kế theo hướng module hóa, trong đó mỗi thành phần đảm nhiệm một vai trò cụ thể như quản lý API, điều phối hội thoại, lưu trạng thái, tích hợp mô hình ngôn ngữ, truy xuất dữ liệu thời tiết, gợi ý địa điểm và quản lý người dùng.

Bên cạnh việc mô tả kiến trúc, chương này cũng đã làm rõ cách một câu hỏi của người dùng được xử lý trong hệ thống, từ bước tiếp nhận request, kiểm tra session, lưu ngữ cảnh hội thoại, chạy workflow LangGraph cho đến bước sinh phản hồi và lưu lịch sử. Cách tổ chức này thể hiện tư duy xây dựng hệ thống theo quy trình rõ ràng, trong đó mỗi bước đều có chức năng riêng nhưng vẫn liên kết chặt chẽ với nhau để tạo thành một luồng xử lý thống nhất. Đặc biệt, việc kết hợp giữa workflow có kiểm soát và mô hình ngôn ngữ lớn cho thấy nhóm không chỉ chú trọng vào khả năng hiểu ngôn ngữ tự nhiên, mà còn quan tâm đến tính chính xác, tính ổn định và khả năng mở rộng của hệ thống.

Ngoài ra, chương này cũng cho thấy hệ thống không được xây dựng theo hướng chỉ phản hồi đơn lẻ, mà được phát triển như một chatbot có khả năng hội thoại nhiều lượt, có quản lý trạng thái, có

dữ liệu nền, có cấu hình vận hành và có cơ chế lưu vết quá trình sử dụng. Đây là điểm quan trọng vì nó phản ánh tư duy thiết kế một sản phẩm có tính ứng dụng thực tế, thay vì chỉ dừng ở mức minh họa ý tưởng.

Nhìn chung, nội dung của chương 2 đã đặt nền tảng đầy đủ cho việc đánh giá hệ thống ở chương tiếp theo. Từ kiến trúc, dữ liệu, quy trình xử lý đến các mô-đun chức năng chính đều đã được trình bày tương đối rõ ràng, tạo cơ sở để phân tích kết quả hoạt động, ưu điểm, hạn chế và khả năng mở rộng của chatbot trong thực tế.

Chương 3

KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

3.1 Mục tiêu đánh giá cho hệ thống chatbot

Sau khi hoàn thiện hệ thống, nhóm sẽ tiến hành đánh giá mức độ hoạt động dựa trên các tiêu chí sau:

- Kiểm tra tính chính xác của luồng xử lý hội thoại.
- Kiểm tra khả năng nhận diện ý định.
- Kiểm tra khả năng trích xuất địa điểm.
- Kiểm tra phản hồi trong trường hợp đủ và thiếu thông tin.
- Kiểm tra chức năng gợi ý địa điểm.
- Kiểm tra khả năng lưu lịch sử hội thoại.

3.2 Môi trường thử nghiệm

Hệ thống được thử nghiệm trong môi trường cục bộ với Python và các thư viện liên quan. Client có thể là Postman, CLI hoặc giao diện web đơn giản. Dữ liệu thời tiết được truy xuất thông qua WeatherAPI. Mô hình LLM được gọi qua Ollama.

3.3 Kết quả hoạt động theo nhóm chức năng

3.3.1 Chức năng đăng ký và đăng nhập

Khi người dùng thực hiện đăng ký tài khoản, hệ thống có thể lưu thông tin vào SQLite và tạo hồ sơ ban đầu. Khi đăng nhập, hệ thống xác thực người dùng và cho phép tiếp tục tương tác với chatbot.

user_id INTEGER	* username TEXT	* email TEXT	* password_hash TEXT	created_at TIMESTAMP
1	testuser	test@example.com	ef92b778bafe771e89245b8	2026-04-11 10:45:42
2	leelinn	1@gmail.com	8d969eef6ecad3c29a3a629	2026-04-11 10:52:22
3	15haiphongchaoaenhe	15haiphong@sv.gmail.com	8d969eef6ecad3c29a3a629	2026-04-11 11:34:58

Hình 3.1: Hệ thống SQLite

Kết quả cho thấy phần quản lý người dùng hoạt động ổn định trong quy mô thử nghiệm. Đây là điều kiện cần để hệ thống có thể quản lý lịch sử hội thoại gắn với từng người dùng cụ thể.

3.3.2 Chức năng nhận câu hỏi và quản lý session

Hệ thống có thể nhận request chat từ người dùng và gắn request đó với một session hội thoại. Nếu người dùng đang ở giữa một luồng hội thoại nhiều lượt, hệ thống vẫn có thể tiếp tục trạng thái cũ.

Ví dụ:

Hình 3.2: Ví dụ kết quả 1

Trong thử nghiệm, chatbot hiểu được rằng "Ở Hà Nội" là câu trả lời bổ sung cho địa điểm còn thiếu trước đó. Điều này chứng tỏ việc quản lý trạng thái hoạt động hiệu quả.

3.3.3 Chức năng phân loại ý định

Đây là một chức năng rất quan trọng. Nhóm kiểm thử chatbot với nhiều dạng câu hỏi khác nhau:

- Hỏi thời tiết hiện tại.
- Hỏi dự báo.
- Xin gợi ý địa điểm.
- Small talk.

Hình 3.3: Ví dụ kết quả 2

Hình 3.4: Ví dụ kết quả 3

Kết quả cho thấy chatbot phân loại đúng phần lớn các câu hỏi phổ biến. Đặc biệt, khi người dùng dùng cách diễn đạt tự nhiên hoặc ngắn gọn, LLM vẫn hỗ trợ suy luận khá tốt.

3.3.4 Chức năng trích xuất địa điểm

Hệ thống nhận diện địa điểm theo hai bước: đối chiếu dữ liệu địa danh và fallback sang LLM. Cách kết hợp này hoạt động tốt trong nhiều trường hợp:

— Địa điểm phổ biến như Hà Nội, Đà Nẵng, TP. Hồ Chí Minh.

— Địa điểm nằm trong tập dữ liệu `locations.json`.

— Câu hỏi có dạng tự nhiên hơn thay vì chỉ nêu tên địa phương trực tiếp.

Tuy nhiên, một số trường hợp viết sai chính tả quá nhiều hoặc địa danh quá ít phổ biến vẫn có thể làm giảm độ chính xác.

3.3.5 Chức năng truy xuất dữ liệu thời tiết hiện tại

Khi có đầy đủ địa điểm, hệ thống gọi WeatherAPI để lấy thời tiết hiện tại. Phản hồi thường gồm:

— Nhiệt độ.

— Trạng thái trời.

— Độ ẩm.

— Gió hoặc khả năng mưa.

Kết quả cho thấy chatbot có thể biến dữ liệu dạng cấu trúc thành câu trả lời tự nhiên, ví dụ:

> "Hiện tại ở Hà Nội thời tiết khá mát, nhiệt độ khoảng 28 độ C, trời nhiều mây nhẹ và độ ẩm ở mức tương đối cao."

3.3.6 Chức năng truy xuất dự báo thời tiết

Khi người dùng yêu cầu xem thời tiết trong những ngày tới, chatbot có thể:

— Xác định đây là intent dự báo.

— Tính số ngày cần lấy dự báo.

— Truy xuất dữ liệu dự báo từ API.

— Tổng hợp xu hướng thời tiết.

Ví dụ câu hỏi:

Hình 3.5: Ví dụ kết quả 3

Chatbot có thể trả lời theo từng ngày hoặc theo dạng tổng quát như:

> "Thời tiết tại Huế trong 3 ngày tới dự kiến sẽ chủ yếu là mây với nhiệt độ dao động từ 26°C đến 27°C. Độ ẩm ở mức 80 %, và gió sẽ có thể đạt tốc độ lên tới 8.3 km/h."

3.3.7 Chức năng gợi ý địa điểm

Đây là chức năng nâng cao giúp chatbot vượt ra ngoài khuôn khổ hỏi đáp thông tin đơn thuần. Trong thử nghiệm, chatbot có thể đưa ra gợi ý dựa trên điều kiện thời tiết và loại hoạt động.

Ví dụ:

Hình 3.6: Ví dụ kết quả 4

Hệ thống sẽ xem xét trạng thái thời tiết, dữ liệu địa điểm và mô-đun gợi ý để trả lời theo hướng:

> "Nếu thời tiết thuận lợi, bạn có thể ưu tiên các địa điểm ngoài trời như khu tham quan, công viên hoặc một số điểm du lịch gần trung tâm."

Dù bộ gợi ý hiện chưa phải hệ recommender nâng cao, nhưng kết quả này cho thấy chatbot đã có yếu tố hỗ trợ quyết định.

3.4 Bảng tổng hợp kết quả hoạt động

Bảng 3.1: Bảng tổng hợp kết quả hoạt động của hệ thống

STT	Chức năng	Biểu hiện hoạt động	Đánh giá
1	Nhận request chat	Tiếp nhận câu hỏi qua API ổn định	Tốt
2	Quản lý session	Duy trì ngữ cảnh giữa nhiều lượt hỏi đáp	Tốt
3	Phân loại ý định	Nhận biết được weather, forecast, recommendation, small talk	Tốt
4	Trích xuất địa điểm	Nhận diện tốt với địa danh phổ biến và dữ liệu có sẵn	Khá tốt
5	Kiểm tra thiếu thông tin	Hỏi lại khi chưa có địa điểm hoặc thông tin cần thiết	Tốt
6	Gọi dữ liệu thời tiết	Truy xuất thành công thông tin hiện tại từ WeatherAPI	Tốt
7	Gọi dữ liệu dự báo	Lấy dự báo nhiều ngày và tổng hợp được kết quả	Tốt
8	Gợi ý địa điểm	Có khả năng hỗ trợ chọn địa điểm hoặc hoạt động	Khá
9	Sinh phản hồi tự nhiên	Câu trả lời thân thiện hơn dữ liệu thô	Tốt
10	Lưu lịch sử hội thoại	Ghi lại nội dung tương tác vào SQLite	Tốt

3.5 Ưu điểm của hệ thống

Qua quá trình triển khai và thử nghiệm, nhóm nhận thấy hệ thống có những ưu điểm nổi bật sau:

- Kiến trúc rõ ràng, tách lớp tốt.
- Có workflow hội thoại chặt chẽ nhờ LangGraph.
- Kết hợp được giữa logic nghiệp vụ và LLM.
- Có khả năng hỏi lại khi thông tin thiếu.
- Tích hợp được dữ liệu thời tiết thực tế.
- Có thể lưu lịch sử hội thoại.
- Có tiềm năng mở rộng thành trợ lý du lịch hoặc trợ lý cá nhân đơn giản.

3.6 Hạn chế của hệ thống

Dù đạt được nhiều kết quả tích cực, hệ thống vẫn còn một số hạn chế:

- Session hiện chủ yếu lưu trong bộ nhớ nên khó mở rộng nếu triển khai lớn.
- Độ chính xác phụ thuộc một phần vào mô hình LLM.
- Chưa có cơ chế đánh giá định lượng chuyên sâu như accuracy intent hoặc entity extraction.
- Bộ gợi ý địa điểm còn dựa nhiều vào luật và dữ liệu tĩnh.
- Chưa tối ưu với các câu hỏi phức tạp, nhiều địa điểm hoặc nhiều mốc thời gian trong cùng một câu.

3.7 Khả năng mở rộng của hệ thống

Từ nền tảng hiện tại, hệ thống có thể được phát triển thêm theo các hướng:

— Bổ sung nhiều intent hơn như cảnh báo thời tiết xấu.

— So sánh thời tiết giữa nhiều khu vực.

— Tích hợp bản đồ hoặc gợi ý lộ trình.

— Thay SessionManager bằng Redis.

— Tích hợp thêm hồ sơ người dùng để cá nhân hóa gợi ý.

— Xây dựng giao diện web hoàn chỉnh.

3.8 Nhận xét chung về kết quả

Nhìn chung, hệ thống Chatbot_Weather đã thực hiện được phần lớn các mục tiêu đặt ra ban đầu. Chatbot không chỉ trả lời được câu hỏi đơn giản mà còn có khả năng quản lý hội thoại nhiều lượt, xử lý trường hợp thiếu dữ liệu, kết nối với dịch vụ thời tiết thực tế và sinh phản hồi gần gũi hơn với người dùng. Điều này chứng tỏ mô hình kiến trúc được lựa chọn là phù hợp với bài toán và có tính ứng dụng tốt trong thực tế.

3.9 Kết luận chương

Trong chương này, nhóm đã trình bày kết quả triển khai và đánh giá hoạt động của hệ thống Chatbot_Weather trên nhiều phương diện khác nhau. Thông qua các nhóm chức năng chính như đăng ký, đăng nhập, quản lý session, phân loại ý định, trích xuất địa điểm, truy xuất dữ liệu thời tiết, dự báo thời tiết, gợi ý địa điểm và xử lý small talk, có thể thấy hệ thống đã đáp ứng được phần lớn các mục tiêu đặt ra ban đầu.

Kết quả thử nghiệm cho thấy chatbot không chỉ hoạt động được ở mức trả lời câu hỏi đơn lẻ, mà còn có khả năng duy trì ngữ cảnh hội thoại, xử lý các trường hợp thiếu thông tin và tiếp tục trao đổi theo nhiều lượt. Đây là một điểm quan trọng vì nó phản ánh đúng định hướng xây dựng hệ thống theo mô hình hội thoại có trạng thái, thay vì chỉ xử lý từng câu hỏi rời rạc. Bên cạnh đó, việc kết hợp giữa dữ liệu thời tiết thực tế, workflow điều phối bằng LangGraph và mô hình ngôn ngữ lớn đã giúp chatbot vừa đảm bảo được tính logic trong xử lý, vừa tạo ra phản hồi có tính tự nhiên hơn đối với người dùng.

Ngoài những kết quả tích cực, chương này cũng đã chỉ ra một số hạn chế còn tồn tại của hệ thống như độ chính xác vẫn phụ thuộc một phần vào mô hình LLM, bộ gợi ý địa điểm chưa thật sự thông minh và khả năng mở rộng ở quy mô lớn vẫn cần tiếp tục hoàn thiện. Tuy nhiên, những hạn chế này không làm mất đi giá trị của hệ thống hiện tại, mà ngược lại còn cho thấy rõ các hướng cải tiến cụ thể trong tương lai.

Nhìn chung, kết quả triển khai và thử nghiệm cho thấy hệ thống Chatbot_Weather đã được xây dựng theo đúng định hướng ban đầu, có khả năng hoạt động ổn định trong phạm vi bài toán và có tiềm năng phát triển thêm trong các ứng dụng thực tế. Đây là cơ sở quan trọng để chuyển sang chương tiếp theo, nơi nhóm đưa ra kết luận tổng thể về đề tài và định hướng phát triển trong tương lai.

Chương 4

KẾT LUẬN TỔNG THỂ

4.1 Kết luận chung

Đề tài Xây dựng hệ thống Chatbot thời tiết đã hoàn thành mục tiêu nghiên cứu và xây dựng một hệ thống chatbot có khả năng tương tác bằng tiếng Việt với người dùng trong bài toán tra cứu thời tiết. Sản phẩm không chỉ đơn thuần là một chương trình trả lời tự động, mà là một hệ thống có kiến trúc khá hoàn chỉnh, bao gồm nhiều thành phần phối hợp chặt chẽ như API backend, bộ quản lý trạng thái hội thoại, mô hình ngôn ngữ lớn, dịch vụ thời tiết bên ngoài, mô-đun gợi ý và cơ sở dữ liệu người dùng.

Điểm mạnh nổi bật của đề tài là việc tổ chức luồng xử lý hội thoại theo mô hình workflow có trạng thái. Cách tiếp cận này giúp hệ thống tránh được sự rối rắm thường gặp ở chatbot viết theo kiểu điều kiện lồng nhau, đồng thời giúp việc mở rộng và bảo trì về sau trở nên thuận tiện hơn. Việc sử dụng LangGraph đã chứng minh được hiệu quả trong bài toán điều phối đa bước, đặc biệt ở các trường hợp người dùng chưa cung cấp đầy đủ thông tin và chatbot cần hỏi lại trước khi trả lời.

Ngoài ra, việc tích hợp mô hình ngôn ngữ lớn vào các bước như phân loại ý định, trích xuất địa điểm và sinh phản hồi giúp hệ thống có khả năng giao tiếp tự nhiên hơn. Đây là yếu tố rất quan trọng trong bối cảnh các hệ thống chatbot hiện đại ngày càng hướng tới trải nghiệm người dùng giống đối thoại thực tế. Tuy nhiên, nhóm cũng không để LLM kiểm soát toàn bộ hệ thống mà chỉ dùng nó ở những khâu phù hợp, trong khi các thao tác nghiệp vụ chính vẫn được kiểm soát bằng logic rõ ràng. Điều này giúp hệ thống cân bằng giữa tính linh hoạt và độ tin cậy.

4.2 Giá trị đạt được của đề tài

Thông qua quá trình nghiên cứu và thực hiện, đề tài đã mang lại những giá trị quan trọng trên cả hai phương diện là lý thuyết học thuật và ứng dụng thực tiễn. Về mặt học thuật, dự án là kết quả của việc vận dụng và phối hợp hệ thống các kiến thức chuyên sâu trong lĩnh vực Khoa học dữ liệu và Trí tuệ nhân tạo. Cụ thể, nhóm đã nghiên cứu và thực thi hóa thành công các kỹ thuật xử lý ngôn ngữ tự nhiên hiện đại, thiết kế kiến trúc hệ thống hội thoại dựa trên các mô hình ngôn ngữ lớn (LLM), đồng thời làm chủ các kỹ năng về xây dựng kiến trúc Backend API, quản lý cơ sở dữ liệu và tích hợp các dịch vụ dữ liệu ngoại vi. Việc kết hợp đa dạng các mảng kiến thức này không chỉ giúp củng cố nền tảng chuyên môn mà còn rèn luyện tư duy thiết kế hệ thống toàn diện cho các thành viên.

Về mặt thực hành, đề tài đã hiện thực hóa thành công một sản phẩm chatbot hoàn chỉnh, có khả năng vận hành ổn định trong môi trường thực tế. Hệ thống cho phép tiếp nhận và xử lý các yêu cầu (request) thông qua giao diện API, phân tích ngữ cảnh câu hỏi của người dùng để truy xuất dữ liệu

thời tiết thực tế và tự động sinh ra các phản hồi thông minh. Thành quả này không chỉ dừng lại ở một mô hình thực nghiệm mà còn là minh chứng rõ nét cho việc chuyển hóa hiệu quả các tri thức lý thuyết nền tảng thành một ứng dụng công nghệ có giá trị sử dụng cụ thể, đáp ứng nhu cầu thực tế của người dùng trong việc tra cứu thông tin và hỗ trợ ra quyết định.

4.3 Những điểm thành công chính

Hệ thống đã đạt được những kết quả khả quan thông qua việc thiết lập một kiến trúc chatbot minh bạch và khoa học. Thành công tiên quyết nằm ở việc tích hợp đồng bộ các công nghệ hiện đại bao gồm FastAPI, LangGraph, các mô hình ngôn ngữ lớn (LLM), hệ quản trị SQLite và WeatherAPI vào một chỉnh thể thống nhất. Về mặt tính năng, chatbot không chỉ dừng lại ở việc xử lý đa dạng các nhóm ý định (intent) của người dùng mà còn thể hiện sự thông minh trong việc duy trì hội thoại nhiều lượt nhờ khả năng quản lý trạng thái chặt chẽ. Đặc biệt, hệ thống đã hình thành cơ chế phản hồi tương tác khi chủ động hỏi lại người dùng để làm rõ các thông tin còn thiếu, đồng thời mở rộng giá trị sử dụng thông qua mô-đun gợi ý địa điểm linh hoạt theo điều kiện thời tiết thực tế.

4.4 Những hạn chế cần khắc phục

Mặc dù đạt được những bước tiến quan trọng, hệ thống vẫn tồn tại một số phương diện cần được cải thiện để tối ưu hóa trải nghiệm người dùng. Hiện tại, dự án vẫn thiếu hụt một bộ dữ liệu kiểm thử chuẩn hóa để thực hiện các đánh giá định lượng về độ chính xác và hiệu quả xử lý. Về mặt kỹ thuật, hệ thống chưa được tinh chỉnh để đáp ứng tốt hiệu năng khi phục vụ quy mô người dùng lớn và các phản hồi vẫn mang tính chất tổng quát, chưa thực sự cá nhân hóa dựa trên hồ sơ riêng biệt của từng cá nhân. Bên cạnh đó, việc thiếu vắng một giao diện người dùng (frontend) hoàn chỉnh và khả năng xử lý hạn chế đối với các truy vấn quá dài hoặc chứa thực thể phức tạp cũng là những rào cản cần được tháo gỡ trong các giai đoạn tiếp theo.

4.5 Hướng phát triển tương lai

Để nâng cao năng lực vận hành và tính ứng dụng thực tiễn, định hướng phát triển tương lai của đề tài sẽ tập trung vào việc nâng cấp hạ tầng kỹ thuật và làm phong phú thêm hệ sinh thái tính năng. Về mặt hệ thống, việc thay thế cơ chế quản lý phiên làm việc trong bộ nhớ bằng các giải pháp phân tán như Redis sẽ được ưu tiên để tăng cường khả năng mở rộng. Song song với đó, chatbot sẽ được bổ sung các nhóm ý định chuyên sâu như cảnh báo thiên tai, nhắc nhở thời tiết định kỳ và tích hợp thêm các nguồn dữ liệu ngoại vi về bản đồ, giao thông hay chất lượng không khí. Nhóm cũng hướng tới việc ứng dụng kỹ thuật học máy để xây dựng bộ gợi ý thông minh hơn, tối ưu hóa quy trình xử lý prompt nhằm giảm chi phí vận hành LLM, và hoàn thiện giao diện ứng dụng trên nền tảng Web hoặc Mobile để tiếp cận gần hơn với người dùng cuối.

4.6 Tổng kết cuối cùng

Nhìn chung, đề tài đã cho thấy việc kết hợp giữa kiến trúc hội thoại có trạng thái, mô hình ngôn ngữ lớn và các nguồn dữ liệu bên ngoài là một hướng tiếp cận phù hợp trong xây dựng chatbot chuyên biệt. Đối với bài toán thời tiết, cách tổ chức này giúp hệ thống không chỉ trả lời được các câu hỏi cơ bản, mà còn có khả năng duy trì ngữ cảnh hội thoại, xử lý trường hợp thiếu thông tin, truy xuất dữ liệu thực tế và sinh phản hồi theo hướng tự nhiên hơn đối với người dùng.

Thông qua toàn bộ quá trình triển khai, hệ thống đã thể hiện được một tư duy xây dựng tương đối rõ ràng: bắt đầu từ phân tích bài toán, chuẩn bị dữ liệu, thiết kế kiến trúc, tổ chức workflow xử lý, tích hợp các mô-đun chức năng, đến kiểm thử và đánh giá kết quả hoạt động. Điều này cho thấy đề tài không chỉ có giá trị ở mặt sản phẩm đầu ra, mà còn phản ánh được cách tiếp cận bài bản trong việc phát triển một hệ thống chatbot có khả năng ứng dụng thực tế.

Mặc dù hệ thống hiện tại vẫn còn một số hạn chế và mới dừng ở mức phát triển ban đầu, nhưng những kết quả đạt được đã tạo ra một nền tảng tương đối tốt để tiếp tục mở rộng trong tương lai. Từ nền tảng này, chatbot có thể được phát triển thêm theo hướng thông minh hơn, linh hoạt hơn và có khả năng hỗ trợ sâu hơn trong các bài toán liên quan đến trợ lý số, tư vấn thông tin hoặc các hệ thống hội thoại chuyên biệt khác. Có thể nói, đề tài đã hoàn thành tốt mục tiêu đặt ra và là cơ sở quan trọng cho những hướng nghiên cứu, cải tiến và ứng dụng tiếp theo.

TÀI LIỆU THAM KHẢO

[1] FastAPI Documentation, *FastAPI Framework*, truy cập từ: https://fastapi.tiangolo.com/

[2] LangGraph Documentation, *Building Stateful and Multi-Actor Applications with LLMs*, truy cập từ: https://langchain-ai.github.io/langgraph/

[3] LangChain Documentation, *LangChain Python Documentation*, truy cập từ: https://python.langchain.com/

[4] Ollama Documentation, *Run Large Language Models Locally*, truy cập từ: https://ollama.com/

[5] WeatherAPI Documentation, *WeatherAPI Developer Docs*, truy cập từ: https://www.weatherapi.com/docs/

[6] SQLite Documentation, *SQLite Official Documentation*, truy cập từ: https://www.sqlite.org/docs.html

[7] Python Software Foundation, *Python Documentation*, truy cập từ: https://docs.python.org/3/

[8] Jurafsky, D. và Martin, J. H., *Speech and Language Processing*, tài liệu tham khảo nền tảng về xử lý ngôn ngữ tự nhiên và hệ thống hội thoại.

[9] Manning, C. D. và Schütze, H., *Foundations of Statistical Natural Language Processing*, MIT Press.

[10] Russell, S. và Norvig, P., *Artificial Intelligence: A Modern Approach*, Pearson.

THÔNG TIN MÃ NGUỒN

Toàn bộ mã nguồn, cấu trúc hệ thống và dữ liệu huấn luyện cho đề tài **"Chatbot hỗ trợ thời tiết và du lịch ứng dụng VinaLLaMA"** được lưu trữ công khai trên nền tảng GitHub.

Quét mã QR để truy cập Repository

— **Repository:** https://github.com/lqlinhkhdl/nhom-chatbot-weather

— **Công nghệ sử dụng:** FastAPI, LangGraph, LangChain, VinaLLaMA, SQLite.

— **Trạng thái:** Đang phát triển và cập nhật (v1.0).

Lưu ý: Để chạy được hệ thống, vui lòng tham khảo tệp `README.md` trong thư mục gốc của mã nguồn.